

INSTITUTO FEDERAL FLUMINENSE
CURSO DE ENGENHARIA [VALIDAR COM O AUTOR]

WILLIAM DA SILVA VIANNA

MODELAGEM 3D PROCEDIMENTAL DO LOTUS
ELISE 111S EM BLENDER
AUTOMATIZADA POR MODEL CONTEXT
PROTOCOL (MCP)

CAMPOS DOS GOYTACAZES – RJ

2026

WILLIAM DA SILVA VIANNA

**MODELAGEM 3D PROCEDIMENTAL DO LOTUS
ELISE 111S EM BLENDER
AUTOMATIZADA POR MODEL CONTEXT
PROTOCOL (MCP)**

Monografia apresentada ao Curso de Engenharia
[VALIDAR COM O AUTOR] do Instituto Federal
Fluminense, como requisito parcial para obtenção do
título de Engenheiro [VALIDAR COM O AUTOR].

Área de concentração: Computação Gráfica e Automação
de Pipelines de Modelagem.

Autor e orientador: William da Silva Vianna (o próprio
autor), Instituto Federal Fluminense.

CAMPOS DOS GOYTACAZES – RJ

2026

WILLIAM DA SILVA VIANNA

**MODELAGEM 3D PROCEDIMENTAL DO LOTUS
ELISE 111S EM BLENDER
AUTOMATIZADA POR MODEL CONTEXT
PROTOCOL (MCP)**

Monografia apresentada ao Curso de Engenharia [VALIDAR
COM O AUTOR] do Instituto Federal Fluminense, como
requisito parcial para obtenção do título de Engenheiro
[VALIDAR COM O AUTOR].

Aprovada em: _____

William da Silva Vianna – Autor e Orientador
Instituto Federal Fluminense

Prof. [VALIDAR COM O AUTOR] – Membro da
banca
[VALIDAR COM O AUTOR]

Prof. [VALIDAR COM O AUTOR] – Membro da
banca
[VALIDAR COM O AUTOR]

Aos que entendem a modelagem como engenharia: onde cada vértice é uma decisão documentada e cada superfície, uma hipótese testável.

AGRADECIMENTOS

Agradeço ao Instituto Federal Fluminense pela formação e pela infraestrutura que tornaram este trabalho possível, e ao corpo docente do Curso de Engenharia pelo rigor que moldou a maneira como este projeto foi conduzido.

Registro que este trabalho foi conduzido com a orientação do próprio autor, William da Silva Vianna, que acumulou as funções de orientando e orientador ao longo de todo o desenvolvimento — arranjo que, se por um lado concentrou as decisões técnicas, por outro exigiu que cada uma delas fosse registrada de forma explícita neste documento.

A Victor Vianna, meu sobrinho, pela colaboração com a ideia da modelagem. A sugestão de tomar o Lotus Elise 111S como objeto de estudo foi dele, e sem ela este trabalho não existiria na forma que assumiu.

À comunidade de software livre, em especial aos mantenedores do Blender, cuja documentação pública e código-fonte aberto permitiram que cada decisão deste trabalho fosse verificada na fonte.

“O que não está no script não existe.”

RESUMO

Este trabalho descreve a construção de um modelo tridimensional do automóvel esportivo Lotus Elise 111S (Series 1) integralmente por código, sem utilização de malhas de terceiros, e a integração desse processo a um agente de inteligência artificial por meio do *Model Context Protocol* (MCP). O problema abordado é a ausência de um pipeline reprodutível que una especificação textual, referências fotográficas e edição tridimensional interativa: na prática industrial, ajustes de malha tendem a se perder em sessões manuais não versionadas. A pesquisa adota abordagem de engenharia reversa visual: um conjunto de seis fotografias do veículo real é tratado como *ground truth*; dessas referências derivam-se dimensões de envelope, estações de casco e parâmetros de materiais, formalizados em especificação própria segundo o método de Desenvolvimento Orientado por Especificação. A geometria é gerada por um *loft* de 45 estações transversais com fundo modulado para produzir arcos de roda reais sem operações booleanas, garantindo malha quadrangular dominante e ausência de n-gons. Os materiais empregam o *Principled BSDF* com pintura automotiva multicamada, *clear coat* e micro-flocos metálicos. A verificação é visual e comparativa, com registro de aceite por critério. Os resultados obtidos totalizam 130 objetos de malha, 32.632 faces, zero n-gons e zero arestas não variedade, com envelope de $3,736\text{ m} \times 1,718\text{ m} \times 1,117\text{ m}$, contra alvos de $3,726\text{ m}$, $1,718\text{ m}$ e $1,117\text{ m}$. Nove renders de verificação e cinco imagens de galeria foram produzidos no motor EEVEE. Dos nove critérios de aceitação, sete foram considerados plenamente atendidos e dois, parcialmente: a silhueta carece da linha de caráter lateral viva e o tom da pintura ficou ligeiramente mais escuro que a referência. Conclui-se que a combinação entre especificação formal, geometria procedimental e automação via MCP produz um artefato auditável, reconstruível por um único comando e adequado à verificação independente.

Palavras-chave: Model Context Protocol; Blender; modelagem procedimental; subdivisão de superfícies; renderização fisicamente baseada; engenharia reversa visual.

ABSTRACT

This work describes the construction of a three-dimensional model of the Lotus Elise 111S (Series 1) sports car entirely through code, without third-party meshes, and the integration of that process with an artificial-intelligence agent through the *Model Context Protocol* (MCP). The problem addressed is the lack of a reproducible pipeline that unifies textual specification, photographic references and interactive three-dimensional editing: in industrial practice, mesh adjustments tend to be lost in unversioned manual sessions. The research adopts a visual reverse-engineering approach: a set of six photographs of the real vehicle is treated as ground truth; envelope dimensions, hull stations and material parameters are derived from these references and formalised in a specification following the Specification-Driven Development method. Geometry is generated by a 45-station cross-section loft whose bottom is modulated to produce real wheel arches without boolean operations, ensuring a quad-dominant mesh with no n-gons. Materials employ the *Principled BSDF* with multi-layer automotive paint, clear coat and metallic micro-flakes. Verification is visual and comparative, with acceptance recorded per criterion. The results total 130 mesh objects, 32,632 faces, zero n-gons and zero non-manifold edges, with an envelope of $3.736\text{ m} \times 1.718\text{ m} \times 1.117\text{ m}$ against targets of 3.726 m , 1.718 m and 1.117 m . Nine verification renders and five gallery images were produced with the EEVEE engine. Of the nine acceptance criteria, seven were fully met and two partially: the silhouette lacks the sharp character line and the paint tone came out slightly darker than the reference. It is concluded that the combination of formal specification, procedural geometry and MCP-driven automation yields an auditable artefact, reconstructible with a single command and suitable for independent verification.

Keywords: Model Context Protocol; Blender; procedural modelling; surface subdivision; physically based rendering; visual reverse engineering.

LISTA DE FIGURAS

1.1	Fluxo metodológico iterativo do trabalho. Os desvios identificados na comparação visual retornam ao script de construção, e não a ajustes manuais na cena.	5
4.1	Arquitetura de três camadas. O agente (cliente MCP) não manipula a interface do Blender: ele invoca ferramentas cujo efeito é a execução de código.	23
4.2	Corpus de referência (<i>ground truth</i> visual). Da esquerda para a direita e de cima para baixo: frontal, lateral, traseira, traseira em três-quartos, superior e três-quartos com a pintura azul.	24
5.1	Estrutura do material de pintura. Os três caminhos — cor base, micro-relevo e verniz — são independentes e se combinam no nó de sombreamento.	34
5.2	Imagem de apresentação com rig dramático temporário: luzes de recorte laterais, elementos ópticos acesos e brilho via nó de <i>glare</i>	46
5.3	Ciclo de chamadas do protocolo durante um ciclo de construção e inspeção. Cada chamada é uma mensagem serializada, o que constitui o registro auditável do processo.	46
6.1	Vista frontal: referência (esquerda) e render (direita).	51
6.2	Vista lateral: referência (esquerda) e render (direita). A silhueta geral coincide; a linha de caráter lateral está ausente.	52
6.3	Vista traseira: referência (esquerda) e render (direita).	52
6.4	Vista superior: referência (esquerda) e render (direita).	53
6.5	Vista em três-quartos: referência (esquerda) e render (direita). Reflexos contínuos; tom ligeiramente mais escuro no render.	54
6.6	Detalhe da roda dianteira: seis raios, tampa central, disco e pinça.	54
6.7	Vistas em material neutro, para leitura de forma sem influência de cor e verniz.	55

LISTA DE TABELAS

4.1	Dimensões de referência adotadas e origem de cada valor	25
5.1	Amostra de estações do casco (valores em metros)	36
6.1	Tempos de render da etapa de verificação (EEVEE, 64 amostras, 1500×1000)	48
6.2	Resumo numérico do build	49
6.3	Envelope dimensional: alvo, valor medido e desvio	50
6.4	Objetos de malha por grupo funcional	51
A.1	Tabela completa das estações do casco (metros)	68
B.1	Materiais da cena e parâmetros principais	70
C.1	Totais de malha da cena (malha base, sem subdivisão)	71
C.2	Envelope medido e alvo	71
C.3	Objetos de malha por grupo funcional	72
D.1	Módulos do projeto	73

LISTA DE QUADROS

2.1	Comparação entre representações de superfície	8
2.2	Classes de continuidade geométrica entre superfícies	9
2.3	Regras topológicas adotadas e sua justificativa	10
3.1	Posicionamento em relação às linhas de pesquisa revisadas	21
4.1	Ambiente computacional e versões	23
4.2	Critérios de aceitação e método de verificação	28
5.1	Módulos do projeto e responsabilidade	30
5.2	Funções principais da biblioteca de helpers	32
5.3	Materiais definidos e estratégia de sombreado	34
5.4	Câmeras do conjunto e parâmetros principais	44
6.1	Origem dos triângulos e das arestas de borda	49
6.2	Composição das malhas principais	49
6.3	Resultado da verificação por critério de aceitação	55
8.1	Objetivos específicos, resultado e evidência	61

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
ADR	<i>Architecture Decision Record</i> (registro de decisão de arquitetura)
API	<i>Application Programming Interface</i> (interface de programação de aplicações)
BSD	<i>Bidirectional Scattering Distribution</i> (distribuição bidirecional de espalhamento)
CAM	<i>Computer-Aided Manufacturing</i> (manufatura assistida por computador)
CAD	<i>Computer-Aided Design</i> (projeto assistido por computador)
CA	Critério de aceitação
DCC	<i>Digital Content Creation</i> (criação de conteúdo digital)
DRC	<i>Design Rule Check</i> (verificação de regras de projeto)
EEVEE	<i>Extra Easy Virtual Environment Engine</i> (motor de render do Blender)
FR	Requisito funcional
GLTF	<i>Graphics Language Transmission Format</i>
GPU	<i>Graphics Processing Unit</i> (unidade de processamento gráfico)
HDR	<i>High Dynamic Range</i> (alta faixa dinâmica)
IOR	<i>Index of Refraction</i> (índice de refração)
JSON	<i>JavaScript Object Notation</i>
LLM	<i>Large Language Model</i> (modelo de linguagem de grande porte)
MCP	<i>Model Context Protocol</i>
NURBS	<i>Non-Uniform Rational B-Splines</i>
NFR	Requisito não funcional
PBR	<i>Physically Based Rendering</i> (renderização fisicamente baseada)
PNG	<i>Portable Network Graphics</i>

RPC	<i>Remote Procedure Call</i> (chamada de procedimento remoto)
SDD	<i>Specification-Driven Development</i> (desenvolvimento orientado por especificação)
sRGB	<i>standard RGB</i> (espaço de cor padrão)
UI	<i>User Interface</i> (interface de usuário)
UV	Mapeamento de coordenadas de textura

LISTA DE SÍMBOLOS

w	meia-largura máxima da seção transversal (m)
w_t	meia-largura do topo da seção transversal (m)
z_b	altura do fundo da seção (m)
z_m	altura em que ocorre a largura máxima (m)
z_s	altura do ombro ou cintura (m)
z_{te}	altura da aresta superior (m)
z_t	altura do centro do topo (m)
n	número de vértices de uma face
θ	ângulo de incidência ou de visada (rad)
f_0	refletância em incidência normal
α	rugosidade de microfacetas adimensional
F	termo de Fresnel
G	termo de sombreamento geométrico
D	função de distribuição de normais
L_o	radiância de saída ($\text{W sr}^{-1} \text{m}^{-2}$)
L_i	radiância incidente ($\text{W sr}^{-1} \text{m}^{-2}$)
ρ	função de distribuição de refletância bidirecional

SUMÁRIO

RESUMO	i
ABSTRACT	ii
Lista de Figuras	iii
Lista de Tabelas	iv
LISTA DE ABREVIATURAS E SIGLAS	vi
LISTA DE SÍMBOLOS	viii
1 Introdução	1
1.1 Contextualização	1
1.2 Problema de pesquisa	1
1.3 Hipótese	2
1.4 Justificativa	2
1.5 Objetivos	3
1.5.1 Objetivo geral	3
1.5.2 Objetivos específicos	3
1.6 Delimitação do escopo	4
1.7 Visão geral da metodologia	4
1.8 Organização do trabalho	4
2 Fundamentação teórica	7
2.1 Representação de superfícies	7
2.1.1 Malhas poligonais	7
2.1.2 Representações paramétricas: Bézier e NURBS	7
2.1.3 Critérios de escolha	8
2.2 Subdivisão de superfícies	8
2.2.1 O esquema de Catmull-Clark	8
2.2.2 Esquemas triangulares e variantes	9
2.2.3 Continuidade e sua leitura visual	9
2.2.4 <i>Loops</i> de suporte e de proximidade	9
2.3 Integridade topológica	10

2.3.1	Quads, triângulos e n-gons	10
2.3.2	Validade: variedade, orientação e normais	10
2.4	Modelos de sombreamento	11
2.4.1	A equação de render	11
2.4.2	Modelos empíricos	11
2.4.3	Microfacetas	11
2.4.4	Fresnel e a aproximação de Schlick	12
2.4.5	Anisotropia	12
2.5	Renderização fisicamente baseada	12
2.5.1	Princípios	12
2.5.2	A formulação de Disney e sua adoção industrial	13
2.5.3	Transmissão e refração rugosa	13
2.6	A pintura automotiva como problema de sombreamento	13
2.6.1	Estrutura em camadas	13
2.6.2	Flocos metálicos	13
2.6.3	Verniz	14
2.7	Gerenciamento de cor e mapeamento de tons	14
2.8	Motores de render em tempo real com traçado de raios	14
2.9	Geometria procedimental	15
2.10	Protocolos de contexto para agentes	15
2.10.1	Do modelo de linguagem ao uso de ferramentas	15
2.10.2	JSON-RPC e o Model Context Protocol	15
2.10.3	Implicações para pipelines de conteúdo tridimensional	16
2.11	Engenharia reversa visual de dimensões	16
2.12	Síntese do capítulo	17
3	Trabalhos relacionados	18
3.1	Modelagem procedimental e gramáticas de forma	18
3.2	Engenharia reversa de modelos geométricos	18
3.3	Reconstrução tridimensional a partir de imagens	19
3.4	Agentes de inteligência artificial que operam ferramentas	19
3.5	Verificação e validação em artefatos digitais	20
3.6	Posicionamento do trabalho	20
4	Materiais e métodos	22
4.1	Natureza e classificação da pesquisa	22
4.2	Ambiente computacional	22
4.3	Arquitetura do pipeline e mediação por MCP	22
4.4	Método de especificação adotado	23
4.5	Corpus de referência	24

4.6	Derivação das dimensões de referência	25
4.7	Estratégia de construção geométrica	26
4.7.1	Opção adotada: superfície de revolução generalizada	26
4.7.2	Arcos de roda sem operações booleanas	26
4.7.3	Tampas de extremidade	26
4.7.4	Recortes e rebaixos reais	26
4.8	Estratégia de materiais	27
4.9	Estratégia de iluminação e câmeras	27
4.10	Procedimento experimental	27
4.11	Critérios de aceitação	28
4.12	Reprodutibilidade	28
4.13	Ameaças à validade	29
5	Desenvolvimento	30
5.1	Visão geral do código	30
5.1.1	Detecção da raiz do projeto	30
5.1.2	Recarregamento do módulo em sessão longa	31
5.2	Biblioteca de helpers	32
5.2.1	Conversão de cor	32
5.2.2	Materiais por identificador de tipo de nó	32
5.2.3	Medição de malha	33
5.3	Materiais	33
5.3.1	A pintura multicamada	34
5.3.2	Habilitação da refração	35
5.4	O casco principal	35
5.4.1	Perfil transversal	35
5.4.2	Estações do casco	36
5.4.3	Arcos de roda sem booleanas	37
5.4.4	Tampas de extremidade	38
5.4.5	Anéis de retenção	38
5.4.6	Rebaixos reais	39
5.5	Cabine, vidros e interior	39
5.6	Rodas	41
5.7	Conjuntos ópticos	41
5.8	Ventilações e aerodinâmica	41
5.9	Emblemas	42
5.10	Nomenclatura, metadados e subdivisão	42
5.11	Relatório de malha	43
5.12	Rig de verificação e câmeras	43
5.13	Configuração de render e mudanças de API	44

5.14	A galeria de apresentação	45
5.15	O ciclo de chamadas via MCP	45
6	Resultados	48
6.1	Protocolo executado e tempos	48
6.2	Resultado do build	48
6.3	Integridade topológica dos corpos principais	49
6.4	Envelope dimensional	50
6.5	Conteúdo da cena	50
6.6	Análise visual comparativa	50
6.6.1	Vista frontal	51
6.6.2	Vista lateral	51
6.6.3	Vista traseira	52
6.6.4	Vista superior	53
6.6.5	Vista em três-quartos e pintura	53
6.6.6	Detalhe da roda	53
6.6.7	Vistas em material neutro	53
6.7	Resultado por critério de aceitação	54
6.8	Síntese dos resultados	55
7	Discussão	56
7.1	Interpretação dos resultados	56
7.2	Sobre os critérios parciais	56
7.2.1	A silhueta e o problema da aresta viva	56
7.2.2	O tom da pintura	57
7.3	O que a mediação por MCP agrega	57
7.4	Comparação com a literatura	58
7.5	Limitações	59
7.6	Implicações	59
7.7	Contribuições	60
8	Conclusão	61
8.1	Retomada do problema	61
8.2	Respostas aos objetivos específicos	61
8.3	Conclusão geral	62
8.4	Trabalhos futuros	62
8.5	Considerações finais	63
	Referências Bibliográficas	64
	A Tabela completa das estações do casco	68

B	Inventário de materiais	70
C	Inventário de objetos e relatório de malha	71
C.1	Totais consolidados	71
C.2	Envelope dimensional	71
C.3	Inventário por grupo funcional	72
C.4	Objetos com arestas de borda	72
C.5	Objetos com triângulos	72
D	Módulos, execução e reprodutibilidade	73
D.1	Estrutura de <i>software</i>	73
D.2	Comandos de reconstrução	73
D.3	Condições de reprodutibilidade	74
D.4	Artefatos de especificação	74
D.5	Divergência identificada durante a redação	74

1 INTRODUÇÃO

1.1 Contextualização

A modelagem tridimensional é, na engenharia contemporânea, uma atividade de projeto e não apenas de ilustração. Um modelo de superfície de um veículo automotivo codifica decisões geométricas — curvaturas, tangentes, dimensões de envelope, folgas — que podem ser verificadas, medidas e reproduzidas. Quando esse modelo nasce de um processo manual e interativo, porém, boa parte dessas decisões permanece implícita: vive na memória de quem operou o *software*, e não em um artefato auditável. É comum que um ajuste de malha bem-sucedido não possa ser reconstruído depois, porque foi feito com o mouse, em uma sequência de gestos que ninguém registrou [1].

Em paralelo, os protocolos de integração entre modelos de linguagem de grande porte e ferramentas externas amadureceram rapidamente. O *Model Context Protocol* (MCP) [2] propõe uma interface padronizada, baseada em mensagens JSON-RPC 2.0 [3], para que um agente de inteligência artificial descubra e invoque ferramentas expostas por um servidor. Um servidor MCP para o Blender permite que um agente execute código Python na sessão viva do programa, inspecione a cena e capture imagens da *viewport*. Isso desloca a unidade de trabalho: deixa de ser o gesto e passa a ser o *script*, que é versionável, revisável e reproduzível.

A confluência desses dois movimentos — formalização da geometria e automação por agentes — abre a possibilidade de tratar a modelagem de um veículo como um problema de engenharia de software: com requisitos identificados, decisões de arquitetura registradas, critérios de aceitação observáveis e evidência de verificação rastreável.

1.2 Problema de pesquisa

O problema que orienta este trabalho pode ser enunciado da seguinte forma:

É possível produzir, de forma integralmente reprodutível e automatizada por um agente de inteligência artificial por meio do Model Context Protocol, um modelo tridimensional de um automóvel esportivo cuja geometria e materiais sejam fiéis a referências fotográficas disponíveis, atendendo simultaneamente a restrições de topologia de malha adequada à subdivisão e a um envelope dimensional verificado?

Do enunciado derivam três tensões técnicas que precisam ser resolvidas conjuntamente. A primeira é a **tensão entre fidelidade e automação**: referências fotográficas são perspectivas, com distorção e oclusão, e não blueprints ortográficos; converter fotografias em parâmetros geométricos exige um método explícito de estimativa. A segunda é a **tensão entre expressividade e restrição topológica**: reproduzir recortes, arcos de roda e entradas de ar em geral se faz com operações booleanas, que produzem n-gons e triângulos em arestas visíveis, incompatíveis com subdivisão de qualidade. A terceira é a **tensão entre realismo físico e custo computacional**: pintura automotiva convincente requer *clear coat*, micro-flocos metálicos e transmissão em vidros, o que demanda um modelo de sombreamento fisicamente plausível [4, 5].

1.3 Hipótese

A hipótese de trabalho é que a combinação de três elementos — (i) uma especificação formal com requisitos e critérios de aceitação observáveis, (ii) geração de geometria exclusivamente por código determinístico, com as restrições topológicas impostas no nível do algoritmo, e (iii) mediação de todas as operações destrutivas por um agente conectado via MCP — permite obter um modelo cuja aderência à referência é comparável à de um fluxo manual equivalente, mas cuja reprodutibilidade e auditabilidade são substancialmente superiores.

1.4 Justificativa

A justificativa do trabalho se apoia em três argumentos.

Reprodutibilidade. Um modelo gerado por script pode ser reconstruído a partir do repositório com um único comando, em qualquer máquina com o mesmo *software*. Isso elimina a dependência de sessões manuais e permite que terceiros verifiquem as decisões geométricas por inspeção do código, e não apenas por observação do resultado final. A prática é reconhecida na literatura de modelagem geométrica como condição para projeto paramétrico e por recursos [6, 7].

Auditabilidade do processo de IA. Quando um agente de inteligência artificial opera uma ferramenta de modelagem, o protocolo de comunicação define exatamente o que foi pedido e o que foi executado. Isso cria um registro natural do processo, alinhado à discussão corrente sobre agentes que usam ferramentas [8, 9, 10]. O MCP, por ser um protocolo aberto e baseado em JSON-RPC, torna esse registro independente do fornecedor do modelo de linguagem.

Formação de engenharia. O exercício de derivar requisitos de um documento informal, convertê-los em critérios observáveis e provar o atendimento por evidência reproduz é a competência central do projeto de engenharia [11, 12]. A escolha de um automóvel

com geometria curva, recortes e acabamento superficial exige que essa competência seja aplicada a um caso em que a qualidade não pode ser alegada, apenas demonstrada.

1.5 Objetivos

1.5.1 Objetivo geral

Produzir, com verificação visual comparativa, um modelo tridimensional do Lotus Elise 111S (Series 1) em escala real, construído inteiramente por código executado em Blender através de um agente operando pelo Model Context Protocol, atendendo a requisitos de fidelidade morfológica, integridade topológica e plausibilidade física dos materiais.

1.5.2 Objetivos específicos

- 1) Extrair das referências fotográficas um conjunto de dimensões de envelope, alturas de cintura e parâmetros de perfil coerentes com a ficha do veículo, registrando explicitamente as incertezas da estimativa.
- 2) Implementar um gerador de casco por *loft* de seções transversais que produza arcos de roda reais sem operações booleanas, mantendo malha quadrângular dominante e zero n-gons.
- 3) Modelar os elementos de identidade do 111S — rodas de seis raios, conjuntos ópticos dianteiro e traseiro, entradas de ar atrás das portas, difusor, emblemas — como geometria real, e não como textura ou mapa de normais.
- 4) Construir uma biblioteca de materiais fisicamente plausível, com pintura automotiva multicamada, *clear coat*, micro-flocos e transmissão em vidros e lentes.
- 5) Estabelecer a cadeia de automação via MCP, de modo que build, verificação e galeria sejam executáveis por um agente sem intervenção manual na interface do Blender.
- 6) Emitir, a cada build, um relatório de malha com contagem de n-gons, arestas não variedade, normais invertidas e envelope dimensional, tornando a verificação automática e não apenas opinativa.
- 7) Comparar os renders das vistas frontal, lateral, traseira, superior e três-quartos com as referências e registrar o resultado por critério de aceitação.

1.6 Delimitação do escopo

A pesquisa delimita-se a um único veículo, em uma única variante e com um único acabamento de cor. Estão fora de escopo, por decisão explícita: suíte de testes automatizados de unidade ou integração; validação automatizada de continuidade por mapas de zebra e isofotas; o *rig* de iluminação de estúdio completo descrito na especificação original do cliente; modelagem detalhada do interior; decais e adesivos; animação; e exportação para formatos de intercâmbio como *FBX* ou *GLTF*.

A verificação é exclusivamente visual e comparativa, com registro de aceite por critério. Essa escolha é coerente com a natureza do artefato: a qualidade de uma superfície automotiva é julgada pela continuidade dos reflexos sob iluminação direcional [13], grandeza que não se reduz a um métrica escalar simples sem um observador humano.

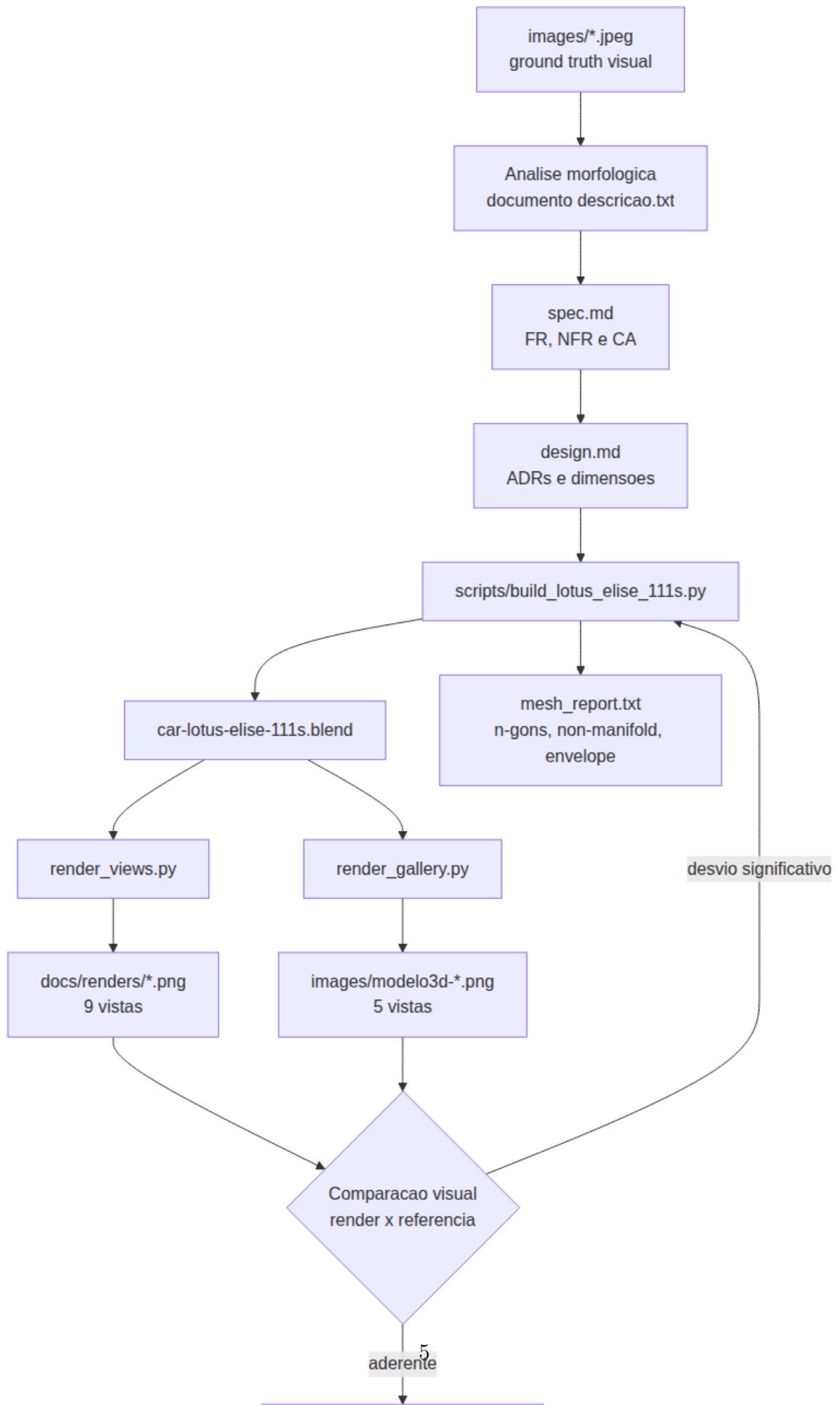
1.7 Visão geral da metodologia

O trabalho segue o método de Desenvolvimento Orientado por Especificação (*Specification-Driven Development*), adaptado a um escopo pequeno e a um único responsável. O método organiza o trabalho em quatro artefatos encadeados: uma *constituição* com os princípios não negociáveis do projeto; uma *especificação* com requisitos funcionais, não funcionais e critérios de aceitação; um *projeto* com decisões de arquitetura registradas; e uma lista de *tarefas* rastreada até o fechamento.

Sobre essa base, o fluxo técnico é iterativo e está representado na Figura 1.1. As referências fotográficas alimentam a especificação, que orienta o script de construção; o script gera o arquivo de cena e um relatório de malha; o arquivo de cena alimenta os scripts de render de verificação e de galeria; os renders são confrontados com as referências; e cada desvio detectado retorna ao script de construção. Todas as etapas destrutivas são mediadas por ferramentas expostas via MCP, de modo que a interface do Blender não é usada como meio de edição.

1.8 Organização do trabalho

Além desta introdução, o texto está organizado em sete capítulos. O Capítulo 2 apresenta a fundamentação teórica: representação de superfícies, subdivisão, sombreamento fisicamente baseado, pintura automotiva, motores de render em tempo real e fundamentos de protocolos de contexto para agentes. O Capítulo 3 discute trabalhos relacionados em quatro frentes: modelagem procedimental de veículos, pipelines de projeto assistido, engenharia reversa a partir de fotografias e agentes que operam ferramentas externas. O Capítulo 4 descreve materiais e métodos, incluindo ambiente, arquitetura de *software*, procedimento experimental e ameaças à validade. O Capítulo 5 detalha a implementação.



O Capítulo 6 apresenta os resultados quantitativos e a análise visual comparativa. O Capítulo 7 interpreta os resultados, discute limitações e implicações. O Capítulo 8 conclui e indica trabalhos futuros. Os apêndices reúnem a tabela de estações do casco, a lista de materiais, a lista de objetos da cena e a referência aos scripts.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo reúne os conceitos necessários para que as decisões técnicas do Capítulo 5 possam ser avaliadas de forma crítica. A exposição parte da representação matemática das superfícies, passa pelo problema da continuidade e da topologia, chega aos modelos de sombreamento e ao gerenciamento de cor e, por fim, trata da automação por script e da mediação por protocolos de contexto para agentes de inteligência artificial.

2.1 Representação de superfícies

2.1.1 Malhas poligonais

A representação mais difundida de uma superfície em computação gráfica é a malha poligonal: um conjunto de vértices, arestas e faces que aproxima a superfície por justaposição de polígonos planos [1]. A malha é, por definição, uma estrutura discreta; sua qualidade depende de como a discretização distribui a curvatura da superfície ideal. Dois aspectos determinam essa qualidade: a *valência* dos vértices (número de arestas incidentes) e a *regularidade* das faces (número de lados).

Malhas poligonais têm vantagens decisivas para tempo real: são diretamente consumidas pela *GPU*, admitem níveis de detalhe variáveis e permitem edição localizada. Sua desvantagem é a ausência de uma representação analítica da curvatura: não existe uma normal contínua em uma aresta viva, e a superfície só é suave na medida em que o sombreamento e a densidade de faces o permitam.

2.1.2 Representações paramétricas: Bézier e NURBS

As representações paramétricas descrevem a superfície como imagem de uma função $\mathbf{S}(u, v)$ de dois parâmetros. As curvas de Bézier e as *B-splines* racionais não uniformes (NURBS) são as mais usadas em projeto assistido por computador [7, 14]. Uma NURBS de grau p é definida por pontos de controle $\mathbf{P}_i$ e pesos w_i :

$$\mathbf{S}(u) = \frac{\sum_{i=0}^n w_i \mathbf{P}_i N_{i,p}(u)}{\sum_{i=0}^n w_i N_{i,p}(u)}, \quad (2.1)$$

em que $N_{i,p}$ são as funções de base *B-spline*. A continuidade da superfície é determinada pela multiplicidade dos nós do vetor de nós: com nós distintos, uma NURBS de grau p é C^{p-1} .

A principal vantagem das NURBS é a continuidade analítica controlável e a exatidão na representação de cônicas [15]. Sua principal desvantagem, no contexto deste trabalho, é a dificuldade de representar *topologia arbitrária*: a estrutura de retalhos (*patches*) exige o controle manual de costuras entre superfícies, e a manipulação programática dessas costuras é significativamente mais complexa que a de uma malha indexada.

2.1.3 Critérios de escolha

O Quadro 2.1 sintetiza a comparação. Para um modelo destinado a renderização fotográfica de um objeto cuja forma não precisa ser enviada a produção por CAM, a malha subdividida oferece o melhor equilíbrio: preserva a manipulação topológica livre e produz, após subdivisão, uma superfície com continuidade suficiente para reflexos automotivos.

Critério	Malha poligonal	NURBS	Malha subdividida
Continuidade	Apenas discreta	Analítica e controlável	Suave exceto em vértices extraordinários
Topologia livre	Sim	Restrita a retalhos	Sim
Custo de render	Baixo	Médio	Médio a alto
Edição por script	Simples	Complexa	Simples
Uso em CAM	Indireto	Direto	Indireto
Representação de recortes	Livre	Limitada	Livre

Fonte: elaborado pelo autor a partir de [1, 7, 16].

Quadro 2.1: Comparação entre representações de superfície

2.2 Subdivisão de superfícies

2.2.1 O esquema de Catmull-Clark

O esquema de Catmull-Clark [17] generaliza as B-splines cúbicas para malhas de topologia arbitrária. Cada iteração de subdivisão executa três passos: insere-se um ponto de face (o centróide), insere-se um ponto de aresta e reposiciona-se cada vértice original por uma média ponderada de seus vizinhos. Para um vértice de valência n , o novo vértice $\mathbf{v}'$ é dado por

$$\mathbf{v}' = \frac{\mathbf{Q} + 2\mathbf{R} + (n - 3)\mathbf{S}}{n}, \quad (2.2)$$

em que $\mathbf{Q}$ é a média dos centróides das faces incidentes, $\mathbf{R}$ é a média dos pontos médios das arestas incidentes e $\mathbf{S}$ é o próprio vértice. Após uma iteração, todas as faces passam a ser quádruplas; iterações seguintes preservam essa propriedade.

A consequência prática é dupla. Primeiro, vértices de valência quatro — a *malha regular* — geram uma superfície C^2 no limite. Segundo, vértices de valência diferente de

quatro, ditos *extraordinários*, geram uma superfície apenas C^1 na vizinhança imediata, mesmo que o restante seja C^2 [16]. Essa descontinuidade localizada é a origem mais comum de artefatos visíveis em reflexos.

2.2.2 Esquemas triangulares e variantes

O esquema de Loop [18] é o análogo triangular, com regras de posicionamento distintas e superfície limite C^2 longe de vértices extraordinários. Existem ainda esquemas interpolantes, como o de Butterfly modificado, que preservam os vértices originais. A escolha por Catmull-Clark neste trabalho se justifica por três motivos: é o esquema padrão do modificador de subdivisão do Blender [19]; produz faces quádruplas, que é a topologia alvo do requisito de malha; e sua avaliação exata foi formalizada por Stam [16], o que permite raciocinar sobre o resultado sem depender de inspeção empírica a cada nível.

2.2.3 Continuidade e sua leitura visual

A classificação clássica da continuidade é resumida no Quadro 2.2. A distinção entre G^1 e G^2 é o ponto central da exigência de qualidade automotiva: dois retalhos tangentes mas com curvaturas diferentes produzem, sob uma fonte luminosa extensa, uma quebra visível na reflexão — a chamada *isofota* descontínua. É por isso que a especificação do cliente proíbe explicitamente a continuidade apenas G^1 [13].

Classe	Propriedade	Efeito visual
G^0	Posição coincidente; tangentes independentes	Aresta viva; reflexo quebrado
G^1	Tangentes coincidentes; curvaturas independentes	Reflexo contínuo, mas com variação abrupta de largura
G^2	Curvaturas coincidentes além das tangentes	Reflexo contínuo e suave; padrão automotivo
C^n	Continuidade paramétrica de ordem n	Implica G^n ; mais restritiva que o necessário

Fonte: elaborado pelo autor a partir de [13, 14].

Quadro 2.2: Classes de continuidade geométrica entre superfícies

2.2.4 Loops de suporte e de proximidade

Em modelagem por subdivisão, a ferramenta para preservar arestas vivas é o *loop* de suporte: um par de cadeias de arestas inseridas paralelamente à aresta que se deseja manter rígida. Quanto mais próximas entre si, mais a superfície limite se aproxima de uma aresta dura. O mesmo mecanismo, aplicado ao redor de recortes, evita o estreitamento excessivo (*pinching*) das faces vizinhas à abertura.

Formalmente, o efeito pode ser entendido como uma restrição do operador de subdivisão: ao concentrar a informação geométrica em uma faixa estreita, o limite deixa de

suavizar aquela região. A técnica é equivalente, em espírito, ao uso de peso de aresta (*crease*), mas é preferível por manter a malha uniformemente quadrangular e o comportamento previsível sob diferentes níveis de subdivisão [1].

2.3 Integridade topológica

2.3.1 Quads, triângulos e n-gons

Uma face com mais de quatro vértices — um n-gon — não admite uma parametrização bilinear natural e, sob subdivisão, tende a produzir resultados dependentes da triangulação interna implícita. Triângulos, por outro lado, são inevitáveis em determinadas terminações e vértices extraordinários, mas concentram curvatura e criam singularidades visíveis quando situados em regiões de curvatura relevante.

Por isso, o requisito de malha quadrangular dominante não é estético: é a condição para que o limite de subdivisão seja previsível. O Quadro 2.3 resume as regras adotadas. A imposição dessas regras no *nível do algoritmo* — e não como etapa de limpeza posterior — é uma decisão de arquitetura discutida no Capítulo 5.

Regra	Justificativa
Zero n-gons	N-gons têm parametrização interna ambígua e degeneram sob subdivisão
Triângulos só em superfície plana oculta	Reduzem a continuidade e criam singularidades no limite
Sem operações booleanas no casco	Booleanas produzem n-gons e triângulos em arestas visíveis
Valência quatro predominante	Vértices extraordinários geram limite apenas C^1
Normais consistentes e externas	Normais invertidas criam artefatos de sombreamento e de refração
Fluxo de arestas alinhado à forma	A direção das cadeias condiciona onde a curvatura é concentrada

Fonte: elaborado pelo autor a partir de [1, 16].

Quadro 2.3: Regras topológicas adotadas e sua justificativa

2.3.2 Validade: variedade, orientação e normais

Além da contagem de faces, uma malha precisa ser uma variedade: cada aresta deve ter no máximo duas faces incidentes. Arestas com três ou mais faces (*não variedade*) tornam ambíguo o conceito de interior e exterior, o que quebra algoritmos de refração e de sombreamento volumétrico. A orientação das faces deve ser consistente, e as normais, apontar para fora.

Essas três propriedades — ausência de n-gons, ausência de arestas não variedade e consistência de normais — formam a base do relatório de malha emitido a cada *build*

deste trabalho (Seção 5.11), o que converte uma propriedade subjetiva em uma asserção verificável por qualquer terceiro.

2.4 Modelos de sombreamento

2.4.1 A equação de render

A luz transportada em uma cena é descrita pela equação de render de Kajiya [20]:

$$L_o(\mathbf{x}, \omega_o) = L_e(\mathbf{x}, \omega_o) + \int_{\Omega} f_r(\mathbf{x}, \omega_i, \omega_o) L_i(\mathbf{x}, \omega_i) (\omega_i \cdot \mathbf{n}) d\omega_i, \quad (2.3)$$

em que L_o é a radiância de saída na direção ω_o , L_e é a radiância emitida, f_r é a função de distribuição de refletância bidirecional (BRDF), L_i é a radiância incidente e Ω é o hemisfério em torno da normal $\mathbf{n}$. Toda a discussão subsequente é uma forma de aproximar a integral de (2.3) ou de parametrizar f_r [21].

2.4.2 Modelos empíricos

O modelo de Phong [22] aproxima a reflexão especular por uma potência do cosseno do ângulo entre a direção de reflexão e a direção de observação. Blinn [23] substituiu a direção de reflexão pelo vetor médio $\mathbf{h}$ entre a direção de luz e a de visada, o que reduz o custo e melhora o comportamento em ângulos rasantes. Apesar de não conservarem energia, ambos os modelos são úteis como referência didática e como caso limite de modelos mais elaborados.

2.4.3 Microfacetas

A teoria de microfacetas modela a superfície como um conjunto estatístico de espelhos minúsculos orientados por uma distribuição de inclinações [24]. A BRDF resultante é

$$f_r(\omega_i, \omega_o) = \frac{k_d}{\pi} + \frac{k_s D(\mathbf{h}) F(\omega_o, \mathbf{h}) G(\omega_i, \omega_o, \mathbf{h})}{4 (\omega_i \cdot \mathbf{n})(\omega_o \cdot \mathbf{n})}, \quad (2.4)$$

em que D é a distribuição de normais das microfacetas, F é o termo de Fresnel e G é o termo de sombreamento mútuo (*shadowing-masking*).

A distribuição Trowbridge-Reitz, popularizada como GGX, é hoje predominante em renderização em tempo real [25, 5]:

$$D_{\text{GGX}}(\mathbf{h}) = \frac{\alpha^2}{\pi [(\alpha^2 - 1)(\mathbf{n} \cdot \mathbf{h})^2 + 1]^2}, \quad (2.5)$$

com α ligado à rugosidade perceptual r por $\alpha = r^2$, escolha que torna a resposta visual aproximadamente linear na rugosidade [5]. A cauda longa de (2.5) é o que confere aos metais polidos sua aparência característica.

O termo de sombreamento é frequentemente aproximado pela forma de Smith, $G = G_1(\omega_i) G_1(\omega_o)$, com G_1 correspondente à mesma distribuição D . A avaliação de G em uma BRDF de microfacetas é o que garante que a energia não seja criada em ângulos rasantes — precisamente os ângulos que dominam a leitura de um para-lama de automóvel.

2.4.4 Fresnel e a aproximação de Schlick

A refletância de uma interface depende do ângulo de incidência. A aproximação de Schlick [26] é a forma padrão:

$$F(\theta) = f_0 + (1 - f_0) (1 - \cos \theta)^5, \quad (2.6)$$

em que $f_0 = ((n_1 - n_2)/(n_1 + n_2))^2$ é a refletância em incidência normal. Para uma superfície dielétrica, $f_0 \approx 0,04$; para metais, f_0 assume a cor do metal. É a variação de F com o ângulo que faz com que uma pintura automotiva escureça no centro da superfície e clareie nas bordas — efeito explorado deliberadamente no material deste trabalho.

2.4.5 Anisotropia

Materiais escovados ou laminados apresentam reflexão dependente da direção tangencial. A extensão anisotrópica clássica é a de Ashikhmin e Shirley [27], baseada em dois expoentes distintos nas direções tangencial e binormal, e a medição sistemática de superfícies reais foi estabelecida por Ward [28]. No presente trabalho, a anisotropia é usada pontualmente no alumínio escovado do bocal de combustível e das saídas de escape.

2.5 Renderização fisicamente baseada

2.5.1 Princípios

A renderização fisicamente baseada (PBR) impõe três disciplinas: conservação de energia, coerência entre os canais de reflexão e difusão, e parametrização a partir de grandezas mensuráveis [4]. A conservação de energia exige

$$k_d = (1 - k_s) (1 - m), \quad (2.7)$$

em que k_s é a fração especular e m é o parâmetro de metalicidade. Sem (2.7), uma superfície metálica ainda refletiria luz difusa, produzindo o aspecto plástico típico de modelos mal parametrizados.

2.5.2 A formulação de Disney e sua adoção industrial

Burley [4] propôs uma parametrização intuitiva para artistas, com parâmetros como *base color*, *metallic*, *roughness*, *specular*, *anisotropy* e *clearcoat*. A difusão de Disney,

$$f_d = \frac{\rho}{\pi} \left(1 + (F_{D90} - 1)(1 - \cos \theta_i)^5 \right) \left(1 + (F_{D90} - 1)(1 - \cos \theta_o)^5 \right), \quad (2.8)$$

reintroduz uma dependência angular que evita o aspecto excessivamente plano da difusão lambertiana pura. Karis [5] adaptou esse modelo ao ambiente de tempo real, e é dessa linhagem que deriva o *Principled BSDF* do Blender [19], nó utilizado em todos os materiais deste trabalho.

2.5.3 Transmissão e refração rugosa

Para vidros e lentes, o modelo de microfacetas é estendido à transmissão por Walter *et al.* [25], que derivam tanto a distribuição quanto o termo de sombreado para o hemisfério oposto. A consequência prática é que um vidro com rugosidade não nula espalha a imagem transmitida — comportamento essencial para diferenciar o policarbonato dos faróis do vidro do para-brisa, ainda que ambos tenham o mesmo índice de refração nominal.

2.6 A pintura automotiva como problema de sombreado

2.6.1 Estrutura em camadas

Uma pintura automotiva real é um sistema de camadas: um *primer* de aderência, uma camada de cor (*base coat*) que contém os pigmentos e, frequentemente, flocos metálicos ou perolados, e uma camada de verniz (*clear coat*) transparente. A luz que chega ao observador atravessou o verniz, foi espalhada pelos pigmentos e voltou a atravessar o verniz. Representar esse caminho corretamente exige mais que uma cor difusa: exige uma camada especular dielétrica sobre um substrato colorido [5, 4].

2.6.2 Flocos metálicos

Os flocos são placas microscópicas de alumínio orientadas preferencialmente paralelas à superfície. Cada floco funciona como um espelho minúsculo, o que produz o brilho granular característico dos metais. Modelar cada floco é inviável; a prática é perturbar a normal ou a rugosidade por um ruído de alta frequência cuja amplitude é pequena o suficiente para ser invisível a distância e perceptível em *close-ups* [5]. A escolha da escala do ruído é, portanto, uma decisão de projeto fotográfico, e não um mero detalhe técnico.

2.6.3 Verniz

O verniz é representado como uma camada dielétrica adicional com seu próprio índice de refração e rugosidade. A presença simultânea das duas camadas especulares — verniz e substrato — produz o efeito de profundidade que distingue uma pintura automotiva de um plástico colorido. Valores típicos de índice de refração do verniz situam-se entre 1,5 e 1,6, e a rugosidade baixa ($\approx 0,02$) é o que permite reflexos nítidos o bastante para revelar descontinuidades de curvatura [5].

2.7 Gerenciamento de cor e mapeamento de tons

O transporte de luz produz radiâncias em uma faixa dinâmica muito superior à que um monitor pode exibir. O mapeamento de tons comprime essa faixa em valores exibíveis, e o operador escolhido altera substancialmente a percepção de cor e contraste [29, 30]. O Blender adota, em versões recentes, o *transform* denominado AgX, que preserva melhor a saturação em altas luzes que o antigo operador Filmic [19].

Uma consequência prática é metodológica: como o operador altera a cor final, a comparação de tom entre render e referência fotográfica não pode ser feita sem considerar o mapeamento de tons. A referência fotográfica foi, ela própria, mapeada pelo processamento da câmera; o render foi mapeado pelo AgX. Parte do desvio de tom registrado no Capítulo 6 é imputável a essa diferença de cadeias, e não ao material.

2.8 Motores de render em tempo real com traçado de raios

A EEVEE, motor padrão do Blender, evoluiu de uma solução puramente *rasterizada* para uma arquitetura com traçado de raios por *GPU*, que combina as duas abordagens: a geometria é rasterizada, e o transporte de luz secundário é estimado por raios lançados a partir de uma estrutura de aceleração [19]. Para o presente trabalho, essa combinação é vantajosa porque o critério de julgamento — a continuidade dos reflexos — depende justamente do transporte secundário, enquanto a velocidade necessária para iterar sobre dezenas de renders só é alcançada em tempo real.

Um último elemento técnico merece registro: o *bloom*, antes um parâmetro do motor, foi migrado para o compositor de imagens, onde é implementado por um nó de *glare* [19]. Essa migração, ocorrida nas versões recentes do Blender, altera a maneira como efeitos volumétricos de luz são obtidos por script — um exemplo concreto de como a automação depende de conhecer a versão exata da API.

2.9 Geometria procedimental

Geometria procedimental é a geração de forma por regras, e não por manipulação direta de vértices. As vantagens são conhecidas: determinismo, parametrização, possibilidade de variar famílias de formas e integração natural com automação [15]. As limitações também: a expressividade fica restrita às regras implementadas, e o projeto de um gerador é tão ou mais trabalhoso que a modelagem manual, sendo compensador apenas quando a reprodutibilidade importa.

No contexto deste trabalho, a asserção que fundamenta a escolha é direta: se a geometria é gerada por código, então a geometria é o código, e ambos podem ser versionados, revisados e reexecutados. A consequência é que o controle de qualidade de malha pode ser imposto por construção, e não verificado *a posteriori*: as tampas de *loft*, por exemplo, são geradas por um algoritmo que só produz quads (Seção 5.4.4), o que torna a ausência de n-gons uma propriedade do método.

2.10 Protocolos de contexto para agentes

2.10.1 Do modelo de linguagem ao uso de ferramentas

Modelos de linguagem de grande porte são, em essência, funções de probabilidade sobre sequências [31, 32]. Para que atuem sobre o mundo, é preciso acoplá-los a ferramentas. As estratégias iniciais consistiam em treinar ou instruir o modelo a emitir chamadas de API em texto [8, 9], com a execução intercalada à deliberação [10, 33]. A recuperação de contexto externo seguiu caminho análogo [34].

Essas abordagens, porém, eram específicas de cada fornecedor: cada integração exigia um adaptador próprio, e o conjunto de ferramentas disponíveis era definido no momento do *prompt*. Faltava um protocolo que separasse a descoberta da execução.

2.10.2 JSON-RPC e o Model Context Protocol

O *Model Context Protocol* [2] propõe exatamente essa separação. Trata-se de um protocolo aberto, sobre JSON-RPC 2.0 [3], que define três papéis: o *host*, que é a aplicação que contém o agente; o *cliente*, que mantém a conexão com um servidor; e o *servidor*, que expõe capacidades. O protocolo padroniza três primitivas: *ferramentas* (*tools*), controladas pelo modelo; *recursos* (*resources*), controlados pela aplicação; e *prompts*, controlados pelo usuário.

O ciclo de vida começa com uma negociação (*initialize*), em que cliente e servidor trocam versão de protocolo e capacidades. Em seguida, o cliente pode descobrir ferramentas (*tools/list*) e invocá-las (*tools/call*). Os transportes padronizados incluem

entrada/saída padrão para processos locais e HTTP com *Server-Sent Events* para servidores remotos.

Do ponto de vista da engenharia, a propriedade relevante é a **serialização completa da interação**: cada chamada de ferramenta é uma mensagem JSON com nome e argumentos, e cada resposta é uma mensagem JSON com conteúdo ou erro. Isso cria um registro do processo independente do modelo de linguagem utilizado — exatamente o que falta em uma sessão manual de modelagem.

2.10.3 Implicações para pipelines de conteúdo tridimensional

Um servidor MCP para o Blender expõe ferramentas como executar código Python, obter informações da cena e capturar a *viewport*. A combinação dessas três capacidades é suficiente para o ciclo completo de trabalho deste projeto: construir (execução), verificar por medição (informações da cena) e verificar por inspeção visual (captura). A análise dessa arquitetura está no Capítulo 5, e a figura 4.1 a resume.

A limitação que decorre do modelo é igualmente clara: o agente não *vê* a cena como um artista a vê; ele vê uma imagem capturada, que é uma projeção de duas dimensões. A inferência sobre profundidade e continuidade a partir dessa imagem é intrinsecamente ambígua [35]. Por isso, a estratégia adotada neste trabalho foi reduzir a dependência da inferência visual: as propriedades de malha são verificadas por medição numérica no relatório, e a inspeção visual fica reservada ao que ela faz melhor — julgar forma, silhueta e continuidade de reflexos.

2.11 Engenharia reversa visual de dimensões

Sem blueprints ortográficos, as dimensões precisam ser estimadas. Duas ferramentas conceituais são relevantes. A primeira é a geometria projetiva: uma fotografia é uma projeção perspectiva, e grandezas medidas na imagem não são proporcionais às grandezas no mundo, exceto sob condições controladas [35]. A segunda é o método de engenharia reversa em projeto de produto, que sistematiza a extração de especificações a partir de um artefato existente [12].

A estratégia adotada combina as duas: quando a literatura e as fontes públicas fornecem a dimensão de envelope, essa dimensão é adotada como restrição dura; as fotografias são usadas para *proporções* relativas — altura de cintura em fração da altura total, posição relativa das rodas, altura do deck — e para validação qualitativa da silhueta. Essa separação é registrada explicitamente para que a origem de cada número seja auditável.

2.12 Síntese do capítulo

A fundamentação estabelece quatro asserções que orientam o restante do trabalho. Primeira: a continuidade G^2 exigida pela especificação só é alcançada com topologia regular e sem n -gons, o que torna a restrição topológica uma condição de qualidade, e não um preciosismo. Segunda: a aparência da pintura automotiva decorre de múltiplas camadas de sombreamento superpostas, o que exclui soluções de cor única. Terceira: o mapeamento de tons altera a cor final e, portanto, condiciona a comparação de tom com a referência. Quarta: um protocolo de contexto para agentes transforma a interação com a ferramenta em um registro serializado, o que é a base da auditabilidade pretendida. O capítulo seguinte situa o trabalho em relação à literatura existente.

3 TRABALHOS RELACIONADOS

O presente trabalho situa-se na interseção de quatro linhas de pesquisa: modelagem procedimental de formas, projetos de engenharia reversa, reconstrução tridimensional a partir de imagens e agentes de inteligência artificial que operam ferramentas externas. Este capítulo percorre cada linha, identifica o que ela oferece e explicita a lacuna que o trabalho pretende ocupar.

3.1 Modelagem procedimental e gramáticas de forma

A ideia de gerar forma por regras antecede a computação gráfica interativa. As gramáticas de forma de Stiny e Gips [36] formalizaram a geração de arranjos geométricos por reescrita, e os sistemas-L de Lindenmayer, popularizados por Prusinkiewicz e Lindenmayer [37], demonstraram que estruturas orgânicas complexas podem ser descritas por um conjunto pequeno de produções.

Na computação gráfica, a modelagem procedimental consolidou-se com trabalhos como o de Parish e Müller [38] para cidades e o de Müller *et al.* [39] para edifícios, nos quais a forma emerge de um conjunto hierárquico de regras aplicadas recursivamente. McCormack *et al.* [40] sistematizaram o paradigma para o design generativo em geral.

A transposição dessa linhagem para o desenho de produto esbarra em uma característica central da estética automotiva: as superfícies são dominadas por curvatura contínua e por relações de proporção perceptuais, difíceis de capturar em regras simbólicas. Trabalhos de modelagem por esboço, como o *Teddy* de Igarashi *et al.* [41], mostram que a interface natural para esse tipo de forma é a manipulação direta de curvas. O presente trabalho não propõe uma gramática nova; propõe que a forma seja *parametrizada por estações*, isto é, descrita por uma tabela de perfis transversais interpolados — uma discretização que preserva o controle artístico e admite geração por código.

3.2 Engenharia reversa de modelos geométricos

A engenharia reversa de modelos geométricos ocupa-se de converter medições do mundo físico em uma representação digital. Várady *et al.* [42] oferecem a introdução clássica ao problema e destacam suas etapas: aquisição, segmentação em regiões compatíveis com primitivas, ajuste de superfícies e imposição de continuidade nas costuras. Otto e Wood [12] situam a prática no contexto mais amplo do desenvolvimento de produto, e Shah e

Mäntylä [6] discutem a integração entre modelagem paramétrica e manufatura.

Esses trabalhos partem, tipicamente, de dados densos — nuvens de pontos de escâner a laser ou medição por coordenadas — e de um objetivo de produção: a superfície reconstruída deve ser executável em CAM. A situação deste trabalho é diferente em dois pontos. Não há dados densos; há seis fotografias em perspectiva. E não há objetivo de manufatura; o objetivo é renderização fotográfica e reprodutibilidade do processo. Isso desloca o problema da fidelidade micrométrica para a fidelidade de *morfologia e proporção*, o que muda as técnicas adequadas.

3.3 Reconstrução tridimensional a partir de imagens

A reconstrução a partir de múltiplas vistas é um campo maduro. A geometria projetiva que a fundamenta está consolidada por Hartley e Zisserman [35]. A reconstrução automática de cenas e estruturas foi popularizada por sistemas como o *Photo Tourism* [43] e pelas implementações modernas de *Structure-from-Motion* [44]. Mais recentemente, campos de radiância neurais [45] e a rasterização gaussiana [46] deslocaram a fronteira para representações implícitas e pontuais, com resultados fotográficos notáveis.

Há, contudo, uma diferença decisiva de objetivo. Esses métodos produzem *reconstruções*: representações que se parecem com o objeto sob os pontos de vista observados. O presente trabalho precisa de um *modelo*: uma malha editável, com topologia controlada, normais coerentes e faces organizadas para subdivisão. Uma reconstrução por fotogrametria gera malha irregular, densa e sem fluxo de arestas — inadequada, por construção, ao requisito de malha quadrângular dominante sem n-gons. As fotografias, aqui, não alimentam um algoritmo de reconstrução automática; alimentam a *decisão humana* sobre parâmetros, e servem de referência de aceitação.

3.4 Agentes de inteligência artificial que operam ferramentas

A capacidade de um modelo de linguagem invocar ferramentas externas foi demonstrada por várias linhas convergentes: o uso de APIs aprendido a partir de documentação [8, 9], a integração massiva de ferramentas [47] e o encadeamento entre raciocínio e ação [10]. A mediação por recuperação de contexto [34] e a coordenação entre múltiplos agentes [48] completam o quadro.

Entretanto, cada uma dessas contribuições resolvia o problema no nível da *aplicação*. Faltava uma camada de protocolo que permitisse a um servidor expor capacidades de forma descobrível e independente do modelo consumidor. O *Model Context Protocol* [2] é a resposta mais difundida a essa lacuna, e sua adoção em ferramentas de desenvolvimento

tem sido rápida. O servidor utilizado neste trabalho, `blender-mcp` [49], expõe o Blender por esse protocolo, com transporte de entrada/saída padrão.

A literatura sobre o uso de MCP em pipelines de conteúdo tridimensional ainda é predominantemente técnica e baseada em relatórios de prática, e não em estudos sistemáticos de qualidade de resultado. O presente trabalho contribui nessa direção ao registrar não apenas o uso do protocolo, mas o efeito desse uso sobre a *reprodutibilidade e a auditabilidade* do artefato geométrico produzido.

3.5 Verificação e validação em artefatos digitais

Em engenharia de software, a verificação responde “o produto foi construído corretamente?” e a validação responde “foi construído o produto correto?”. Transportar essa distinção para conteúdo tridimensional exige cuidado, porque a “correção” de uma superfície automotiva não é uma propriedade binária.

A prática da indústria automotiva resolve isso por inspeção: mapas de zebra e isofotas tornam visível a continuidade dos reflexos, e a avaliação é feita por especialistas treinados [13]. A contribuição metodológica deste trabalho é dupla e deliberadamente assimétrica: aceitar a inspeção visual como evidência de qualidade de superfície, mas *não* aceitá-la como evidência de integridade estrutural. Integridade — n-gons, não variedade, orientação de normais e envelope — é verificada por medição numérica automatizada a cada *build*. É uma posição pragmática: cada grandeza é verificada pelo método adequado à sua natureza.

3.6 Posicionamento do trabalho

O Quadro 3.1 contrasta o presente trabalho com as linhas revisadas. A contribuição reivindicada não é um novo algoritmo de subdivisão, nem um novo método de reconstrução, nem um novo protocolo. É a integração *verificada* de três componentes existentes — especificação formal, geometria procedimental com restrições topológicas impostas por construção, e mediação por agente via protocolo de contexto — em um artefato único cuja qualidade é demonstrada por evidência reproduzível.

Duas ausências de literatura merecem registro explícito, para que não sejam confundidas com omissão. A primeira é a de estudos comparativos controlados sobre a qualidade de modelos 3D produzidos por agentes via MCP: a prática existe, mas a análise sistemática ainda é escassa. A segunda é a de literatura técnica pública e verificável sobre a geometria específica do Lotus Elise 111S em nível de detalhe suficiente para modelagem (estações de superfície, raios de curvatura, tolerâncias). Este trabalho contorna a segunda ausência tratando as fotografias como *ground truth* e registrando as estimativas como tal, conforme discutido na Seção 4.13.

Linha	O que oferece	O que não cobre
Modelagem procedimental	Geração determinística de forma por regras	Morfologia automotiva de curvatura contínua com controle fino de perfil
Engenharia reversa geométrica	Método de reconstrução a partir de medições	Entrada fotográfica esparsa; objetivo de render, não de CAM
Reconstrução por imagens	Fidelidade fotográfica sob vistas observadas	Malha editável com topologia controlada
Agentes com ferramentas	Automação e composição de ações	Verificação de qualidade do artefato produzido

Fonte: elaborado pelo autor (2026).

Quadro 3.1: Posicionamento em relação às linhas de pesquisa revisadas

4 MATERIAIS E MÉTODOS

Este capítulo descreve como o trabalho foi conduzido, com detalhe suficiente para que um terceiro possa reproduzi-lo ou contestá-lo. Registram-se o ambiente, o corpus de entrada, o método de especificação, as decisões de estratégia geométrica e de materiais, o procedimento experimental e as limitações do método.

4.1 Natureza e classificação da pesquisa

Quanto à natureza, a pesquisa é aplicada: não busca uma lei geral, e sim demonstrar que um pipeline específico produz um artefato verificável. Quanto à abordagem, é quantitativa e qualitativa: quantitativa na medição de envelope, contagem de faces e tempos de render; qualitativa na avaliação comparativa das vistas. Quanto ao objetivo, é exploratória com componente experimental controlado. Quanto ao procedimento, é um estudo de caso único, com um único objeto e uma única configuração de software.

Essa configuração tem uma consequência metodológica explícita: como o estudo de caso não permite generalização estatística, todas as afirmações deste trabalho são *sobre o caso*, e as generalizações são apresentadas como hipóteses a testar, não como conclusões.

4.2 Ambiente computacional

O Quadro 4.1 registra as versões exatas. A fixação das versões é parte do método, porque tanto a API do Blender quanto os nomes de parâmetros de configuração de render mudam entre versões, como discutido na Seção 5.13.

A ausência de dependências externas é deliberada. Ela reduz a superfície de falha, garante que o build não dependa de pacotes de terceiros e mantém o tempo de reconstrução baixo o suficiente para permitir iterações rápidas.

4.3 Arquitetura do pipeline e mediação por MCP

A arquitetura tem três camadas (Figura 4.1). Na camada superior, um agente de codificação atua como cliente MCP. Na intermediária, o servidor `blender-mcp` expõe ferramentas e traduz chamadas do protocolo em código Python. Na inferior, o Blender executa esse código na sessão viva e devolve resultados.

Componente	Especificação
Sistema operacional	GNU/Linux (ambiente local de desenvolvimento)
Blender	5.2.1 LTS, unidades métricas, <code>scale_length = 1,0</code>
Motor de render	BLENDER_EEVEE com traçado de raios habilitado
API de modelagem	bpy e bmesh (Python embutido no Blender 3.x)
Servidor MCP	blender-mcp [49], transporte stdio
Lançador do servidor	uvx (/snap/bin/uvx)
Cliente MCP	agentes de codificação em ambiente de desenvolvimento (VS Code)
Persistência	car-lotus-elise-111s.blend (arquivo único)
Diagramas	mermaid-cli com Chrome do sistema
Composição de texto	L ^A T _E X (pdf _l atex) com bib _t ex
Dependências Python externas	nenhuma

Fonte: elaborado pelo autor (2026).

Quadro 4.1: Ambiente computacional e versões

A propriedade arquitetural relevante é que *não existe caminho de edição manual*. Toda modificação da cena é consequência da execução de um script. Se um ajuste não está no script, ele não sobrevive ao próximo build — e essa *propriedade*, formulada como princípio na constituição do projeto, é o que sustenta a reprodutibilidade.

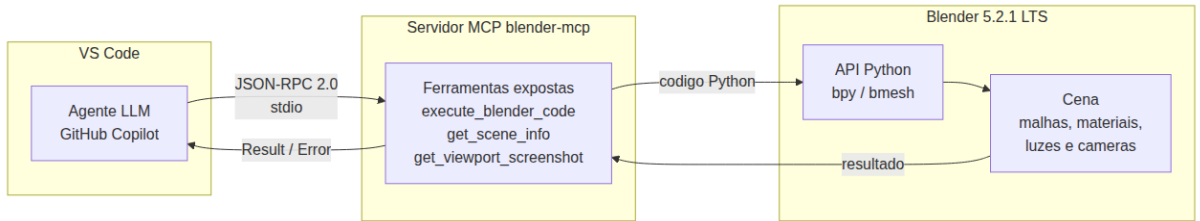


Figura 4.1: Arquitetura de três camadas. O agente (cliente MCP) não manipula a interface do Blender: ele invoca ferramentas cujo efeito é a execução de código.

Fonte: elaborado pelo autor (2026).

4.4 Método de especificação adotado

O trabalho adota uma adaptação enxuta do Desenvolvimento Orientado por Especificação, com quatro artefatos encadeados e versionados:

- 1) **Constituição** — princípios não negociáveis: fidelidade à referência em caso de conflito com o texto; fidelidade dimensional com tolerância de 2 cm; qualidade de malha; reprodutibilidade por script; materiais fisicamente plausíveis; verificação visual documentada; e disciplina de mudança.
- 2) **Especificação** — 21 requisitos funcionais, 4 requisitos não funcionais e 9 critérios de aceitação, cada um com método de verificação declarado.

- 3) **Projeto** — quatro registros de decisão de arquitetura (ADR), dimensões de referência e rejeição explícita de alternativas.
- 4) **Tarefas** — doze tarefas com dependências, condição de conclusão e *gate* de verificação.

A ordem de precedência em caso de conflito é: imagem de referência $\succ$ ficha dimensional $\succ$ texto descritivo. Essa precedência foi escolhida porque as imagens representam o objeto real, enquanto o texto descritivo é uma interpretação do objeto.

4.5 Corpus de referência

O corpus de entrada é um conjunto de seis fotografias do veículo real, apresentado na Figura 4.2. As imagens cobrem as vistas frontal, lateral, traseira (duas tomadas), superior e três-quartos. Uma das imagens mostra um veículo de cor diferente — branco — e foi utilizada exclusivamente para leitura de *forma* — recortes, grelhas, posição relativa de elementos — e não para cor. Essa restrição de uso é registrada como premissa.

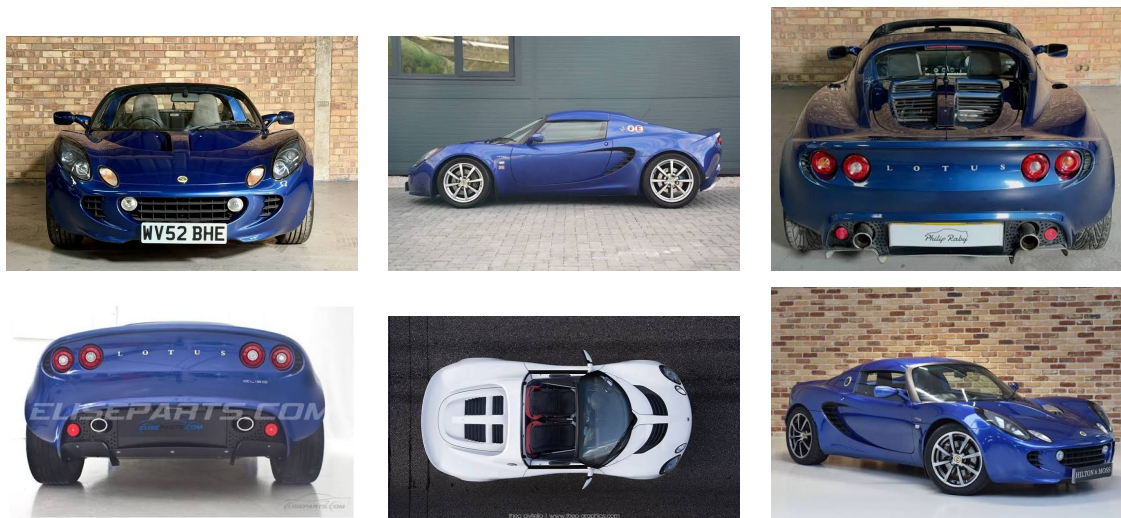


Figura 4.2: Corpus de referência (*ground truth* visual). Da esquerda para a direita e de cima para baixo: frontal, lateral, traseira, traseira em três-quartos, superior e três-quartos com a pintura azul.

Fonte: acervo do projeto (images/).

Duas características do corpus condicionam o método. A primeira é a ausência de blueprints ortográficos: as fotografias têm perspectiva, o que impede a medição direta de proporções [35]. A segunda é a resolução moderada das imagens (entre 516×387 e 723×423 pixels), insuficiente para extrair detalhes micrométricos.

4.6 Derivação das dimensões de referência

O procedimento de derivação seguiu três passos, na ordem de precedência do método:

- 1) **Envelope como restrição dura.** As dimensões de comprimento, largura e altura foram adotadas da especificação do cliente e de ficha pública do Series 1, sendo tratadas como restrição de projeto com tolerância de ± 2 cm.
- 2) **Proporções relativas das fotografias.** As alturas de cintura, de deck e de saia, bem como a posição relativa dos eixos, foram estimadas a partir das vistas lateral e superior, usando elementos de referência internos (diâmetro da roda, altura total) para normalizar.
- 3) **Consistência com a ficha de pneus.** Os raios de pneu, conhecidos pela designação comercial (185/55R15 e 205/50R16), serviram de verificação cruzada: se o raio calculado não fosse compatível com a altura livre estimada, a estimativa era revista.

A Tabela 4.1 consolida os valores adotados. Cada valor é classificado quanto à origem, permitindo que um revisor saiba quais números são restrições externas e quais são estimativas do autor.

Tabela 4.1: Dimensões de referência adotadas e origem de cada valor

Parâmetro	Valor	Origem	Uso
Comprimento total	3,726 m	especificação e ficha	restrição dura
Largura total	1,718 m	especificação e ficha	restrição dura
Altura total	1,117 m	especificação e ficha	restrição dura
Distância entre eixos	2,300 m	ficha do Series 1	restrição dura
Bitola dianteira	1,419 m	ficha do Series 1	restrição dura
Bitola traseira	1,454 m	ficha do Series 1	restrição dura
Pneu dianteiro 185/55R15	raio 0,292 m	designação comercial	derivado
Pneu traseiro 205/50R16	raio 0,306 m	designação comercial	derivado
Altura de cintura	$\approx 0,80$ m	estimativa fotográfica	estimativa
Topo do deck do motor	$\approx 0,92$ m	estimativa fotográfica	estimativa
Altura mínima de saia	$\approx 0,12$ m	estimativa fotográfica	estimativa

Fonte: elaborado pelo autor (2026). As estimativas fotográficas têm incerteza não quantificada, conforme discussão na Seção 4.13.

4.7 Estratégia de construção geométrica

4.7.1 Opção adotada: superfície de revolução generalizada

O casco principal é construído como um *loft* de seções transversais ao longo do eixo longitudinal. Cada estação é um perfil fechado definido por dez pontos de meio-lado, espelhados e unidos, totalizando dezoito segmentos por anel. Os parâmetros de cada estação são sete alturas e larguras características ($w, w_t, z_b, z_m, z_s, z_{te}, z_t$), descritas no Apêndice A. O encadeamento de estações adjacentes por segmentos de reta produz uma malha quadrângular estruturada.

A escolha por esta representação, em vez de NURBS, deve-se a três razões: a parametrização por tabela é diretamente legível e editável; a topologia resultante é quadrângular por construção; e o ajuste de uma estação afeta apenas uma faixa da malha, o que torna a iteração localizada e previsível.

4.7.2 Arcos de roda sem operações booleanas

O recorte do arco de roda é obtido elevando a cota do fundo (z_b) nas estações próximas aos eixos. O efeito é um “túnel” suave sobre o pneu, com a malha quadrângular preservada. A alternativa — diferença booleana entre o casco e um cilindro — foi rejeitada porque produz n-gons e triângulos exatamente sobre a aresta viva do para-lama, que é a região de maior visibilidade do reflexo.

4.7.3 Tampas de extremidade

As extremidades (nariz e cauda) precisam ser fechadas. As operações usuais de preenchimento de grade podem produzir n-gons quando o contorno não é regular. Por isso, o projeto implementa uma tampa por “leque dobrado” que, por construção, só emite quads. Essa decisão é discutida com mais detalhe na Seção 5.4.4.

4.7.4 Recortes e rebaixos reais

As entradas de ar laterais e o alojamento dos faróis são implementados como deslocamento de vértices com função de atenuação (*falloff*), e não como mapas de normais. A profundidade do rebaixo é um parâmetro verificável, e a existência de geometria real sob a superfície assegura que o sombreamento resultante seja geométrico, e não uma ilusão de superfície.

4.8 Estratégia de materiais

Os materiais seguem a formulação de renderização fisicamente baseada discutida na Seção 2, com o nó **Principled BSDF** como bloco central. Três decisões merecem destaque.

Pintura multicamada. O material da carroceria combina uma cor base azul profunda convertida de sRGB para linear, um perturbação de alta frequência para simular os flocos metálicos, e uma camada de verniz com índice de refração e rugosidade próprios. A escurecimento das bordas é obtido por uma rampa de cor controlada pelo *facing* de um nó de peso de camada.

Conversão de espaço de cor. Todas as cores são definidas em sRGB e convertidas para o espaço linear antes de serem atribuídas ao nó. A conversão incorreta é uma fonte comum e sutil de erro de tom, porque o resultado *parece* plausível, mas com saturação sistematicamente deslocada.

Transmissão. Vidros e lentes usam transmissão com rugosidade distinta por aplicação: o policarbonato dos faróis é mais próximo do especular perfeito, enquanto o vidro do para-brisa tem rugosidade ligeiramente maior. A refração por traçado de raios precisa ser habilitada explicitamente nos materiais transparentes para que o efeito apareça no motor EEVEE.

4.9 Estratégia de iluminação e câmeras

O rig de iluminação completo descrito na especificação original — *softboxes*, *light cards*, *HDRI* — foi declarado fora de escopo por decisão do usuário. Em seu lugar, adota-se um rig neutro mínimo, com três luzes de área: uma chave a aproximadamente quarenta e cinco graus, um preenchimento mais fraco do lado oposto e uma luz de recorte traseira. O objetivo do rig neutro não é produzir uma imagem de apresentação, mas revelar continuidade de superfície, tornando visíveis eventuais descontinuidades de curvatura.

A galeria de apresentação usa um rig distinto e temporário, descrito na Seção 5.14, que não é persistido no arquivo de cena.

As câmeras usam distância focal longa (entre 62 mm e 105 mm) e ficam a distâncias entre 7,6 m e 8,2 m do objeto. A escolha por lentes longas minimiza a distorção de perspectiva, aproximando a projeção de uma ortográfica e facilitando a comparação com as fotografias.

4.10 Procedimento experimental

O procedimento é composto por quatro etapas executáveis por script.

- 1) **Build.** Limpação completa da cena e reconstrução de todo o modelo, materiais, rig de verificação e câmeras, seguida de emissão do relatório de malha e gravação

do arquivo de cena.

- 2) **Verificação.** Render de nove vistas em baixa latência: cinco vistas principais, duas vistas três-quartos, um *close* de roda e duas vistas em material neutro (*clay*) para leitura de forma sem influência da cor.
- 3) **Galeria.** Render de cinco imagens de apresentação, incluindo uma com rig dramático e efeito de brilho no compositor.
- 4) **Comparação.** Confronto sistemático entre cada render e a referência correspondente, com registro de aceite por critério.

A etapa de verificação em material neutro merece justificativa. Como a cor e o verniz alteram a leitura da forma, uma vista em *clay* remove essas variáveis e permite julgar silhueta e fluxo de arestas isoladamente. É o análogo prático, para uma malha, do que os mapas de zebra fazem para uma superfície paramétrica.

4.11 Critérios de aceitação

O Quadro 4.2 lista os nove critérios de aceitação e o método de verificação de cada um. A separação entre verificação numérica e verificação visual é explícita, conforme discutido na Seção 3.

ID	Critério	Método
CA-001	Build reconstrói o modelo sem erro	execução do script
CA-002	Envelope dentro de ± 2 cm dos alvos	medição numérica
CA-003	Zero n-gons e normais coerentes nos corpos	relatório de malha
CA-004	Silhueta lateral coerente com a referência	inspeção visual
CA-005	Frente com faróis ovais, grelha e auxiliares	inspeção visual
CA-006	Traseira com lanternas, lettering, escapes e difusor	inspeção visual
CA-007	Pintura com reflexos contínuos, sem facetas	inspeção visual
CA-008	Vista superior com entradas de ar, louvers e roll bar	inspeção visual
CA-009	Nomenclatura e metadados em todos os objetos	inspeção do arquivo

Fonte: elaborado pelo autor (2026).

Quadro 4.2: Critérios de aceitação e método de verificação

4.12 Reprodutibilidade

A reprodução integral do artefato exige apenas o Blender em versão compatível e o repositório de scripts. Os três comandos são:

```
1 blender --background car-lotus-elise-111s.blend --python scripts/  
  build_lotus_elise_111s.py  
2 blender --background car-lotus-elise-111s.blend --python scripts/render_views.py
```

```
3 blender --background car-lotus-elise-111s.blend --python scripts/render_gallery.py
```

A execução via MCP é equivalente e consiste na execução do script dentro da sessão viva do Blender. As duas formas produzem o mesmo resultado, o que é verificado comparando-se o relatório de malha após cada execução. O Capítulo 6 apresenta os tempos medidos.

4.13 Ameaças à validade

Ausência de blueprints. As proporções derivam de fotografias em perspectiva. Mesmo com normalização por elementos de referência internos, há erro sistemático cuja magnitude não pode ser estimada sem uma medição independente. Trata-se da principal limitação de validade interna.

Validação pelo próprio autor. A avaliação visual foi realizada pelo autor do modelo. Embora os renders e o critério estejam registrados, não houve avaliação cega nem múltiplos avaliadores, o que impede estimar a reprodutibilidade da avaliação.

Cor e cadeia de imagem. A referência fotográfica passou pelo processamento da câmera; o render passou pelo mapeamento de tons AgX. Diferenças de tom entre os dois podem decorrer da cadeia de imagem e não do material.

Ambiguidade da avaliação por imagem. A inspeção de uma projeção bidimensional não permite julgar com segurança continuidade de curvatura em profundidade [35]. A mitigação adotada foi a verificação numérica das grandezas estruturais.

Ausência de comparação com um fluxo manual. Não foi executado um segundo modelo pelo método tradicional para servir de controle. Afirmativas comparativas sobre reprodutibilidade são, portanto, argumentativas e não experimentais.

Dependência de versão de software. O comportamento fotométrico e a disponibilidade de parâmetros dependem da versão do Blender. A reprodução em outra versão pode exigir adaptações, discutidas na Seção 5.13.

5 DESENVOLVIMENTO

Este capítulo descreve a implementação. A exposição segue a ordem em que o código constrói a cena: biblioteca de apoio, materiais, casco, cabine, rodas, conjuntos ópticos, ventilações, emblemas, interior, metadados, relatório, rig e render. Os trechos de código apresentados são extratos do código real, com comentários em português substituídos por equivalentes sem acentuação quando necessário para a composição tipográfica.

5.1 Visão geral do código

O projeto tem cinco módulos em `scripts/`, listados no Quadro 5.1. A separação segue um princípio simples: a biblioteca `lotus_common.py` não tem efeitos colaterais — importá-la não altera a cena — o que a torna testável e reutilizável por qualquer script. Os demais módulos têm efeito e são, por isso, explicitamente nomeados como construtores ou renderizadores.

Módulo	Responsabilidade	Efeito colateral
<code>lotus_common.py</code>	Helpers de geometria, materiais, cores e medição	Nenhum
<code>build_lotus_elise_111s.py</code>	Constrói a cena inteira, salva e emite relatório	Sim
<code>render_views.py</code>	Nove renders de verificação	Escreve imagens
<code>render_gallery.py</code>	Cinco imagens de apresentação	Escreve imagens
<code>mesh_report.py</code>	Relatório de malha avulso	Escreve relatório

Fonte: elaborado pelo autor (2026).

Quadro 5.1: Módulos do projeto e responsabilidade

5.1.1 Detecção da raiz do projeto

Um problema concreto e não óbvio precisa ser resolvido antes de qualquer outra coisa: quando um script é executado dentro de uma sessão do Blender, o diretório de trabalho do processo *não* é necessariamente o diretório do projeto. Em ambiente de desenvolvimento com múltiplos projetos abertos, isso faz com que caminhos relativos apontem para o lugar errado — e o modo de falha é silencioso, porque o render simplesmente é gravado no inlugar errado.

A solução adotada é uma função de detecção que sobe a árvore de diretórios a partir de três candidatos e aceita como raiz o primeiro que contenha *simultaneamente* o módulo de helpers e a pasta de imagens de referência:

```
1 def _find_base() -> str:
2     """Project root: requires scripts/lotus_common.py and images/."""
3     candidates = []
4     if bpy.data.filepath:
5         candidates.append(os.path.dirname(bpy.data.filepath))
6     try:
7         candidates.append(os.path.dirname(os.path.abspath(__file__)))
8     except NameError:
9         pass
10    candidates.append(os.getcwd())
11    for start in candidates:
12        node = start
13        for _ in range(5):
14            if os.path.isfile(os.path.join(node, "scripts", "lotus_common.py")) \
15                and os.path.isdir(os.path.join(node, "images")):
16                return node
17            parent = os.path.dirname(node)
18            if parent == node:
19                break
20            node = parent
21    return candidates[0]
```

A exigência dupla — helpers e imagens — é o que torna a detecção robusta a falsos positivos: uma pasta que contenha apenas um dos dois não é aceita como raiz.

5.1.2 Recarregamento do módulo em sessão longa

Em uma sessão longa do Blender, como as conduzidas via MCP, o interpretador Python mantém módulos em cache. Isso significa que editar `lotus_common.py` e reexecutar o build *não* tem efeito, porque o módulo antigo continua carregado. O build resolve isso de forma explícita:

```
1 import importlib
2 import lotus_common as lc
3 importlib.reload(lc)
```

Trata-se de um exemplo prático da diferença entre operar um programa por sua interface e operá-lo por código: no primeiro caso, o ciclo de vida do processo é gerenciado pelo usuário; no segundo, é preciso gerenciá-lo explicitamente.

5.2 Biblioteca de helpers

O Quadro 5.2 resume as funções de `lotus_common.py`. O desenho privilegia funções que retornam objetos nomeados e já finalizados, de modo que o código de construção se leia como uma sequência de intenções, e não como uma sequência de operações de baixo nível.

Função	Papel
<code>srgb_to_linear / rgba</code>	Conversão de cor de sRGB para linear
<code>clear_scene</code>	Limpeza idempotente de objetos e dados órfãos
<code>new_material</code>	Material com Principled BSDF por <code>bl_idname</code>
<code>noise_normal</code>	Ruído de alta frequência ligado à normal
<code>finalize</code>	bmesh para objeto, com material, subdivisão e metadados
<code>_folded_fan_cap</code>	Tampa de laço fechado produzindo apenas quads
<code>quad_caps</code>	Substituição de tampas n-gon por malha quadrangular
<code>tag</code>	Gravação das propriedades <code>cat</code> , <code>part</code> e <code>mat_swap_key</code>
<code>build_loft</code>	Superfície fechada por encadeamento de anéis
<code>revolve</code>	Sólido de revolução a partir de perfil
<code>cylinder / box / sphere</code>	Primitivas com chanfro opcional
<code>extrude_profile</code>	Sólido por extrusão de polígono
<code>text_mesh</code>	Texto convertido em malha
<code>body_ring / cabin_ring</code>	Perfis transversais parametrizados
<code>arch_bottom</code>	Fundo do casco com vãos de roda
<code>world_bounds</code>	Envelope global em coordenadas de mundo
<code>mesh_stats</code>	Contagem de quads, triângulos, n-gons, não variedade e borda

Fonte: elaborado pelo autor (2026).

Quadro 5.2: Funções principais da biblioteca de helpers

5.2.1 Conversão de cor

A conversão de sRGB para linear é implementada segundo a recomendação do *standard*: uma transformação linear na região escura e uma potência fora dela.

```
1 def srgb_to_linear(c: float) -> float:
2     return c / 12.92 if c <= 0.04045 else ((c + 0.055) / 1.055) ** 2.4
```

A função é pequena, mas sua ausência teria consequência sistemática: todas as cores do modelo ficariam saturadas de forma incorreta, e o erro seria difícil de atribuir a uma causa única, porque cada material se desviaria de modo diferente.

5.2.2 Materiais por identificador de tipo de nó

A criação de material busca o nó de sombreamento por seu identificador interno (`bl_idname`) e não pelo rótulo exibido. A razão é prática e foi verificada durante o

desenvolvimento: em uma interface configurada em português, o rótulo do nó aparece traduzido, e o código que busca por nome falha sem que a causa seja evidente.

```
1 def new_material(name: str):
2     mat = bpy.data.materials.new(name)
3     mat.use_nodes = True
4     nt = mat.node_tree
5     nt.nodes.clear()
6     out = nt.nodes.new("ShaderNodeOutputMaterial")
7     bsdf = nt.nodes.new("ShaderNodeBsdfPrincipled")
8     nt.links.new(bsdf.outputs["BSDF"], out.inputs["Surface"])
9     return mat, nt, bsdf
```

A lição generaliza para qualquer automação: identificadores de API são estáveis; rótulos de interface não são. Código que se apoia em rótulos quebra na primeira mudança de idioma ou de versão.

5.2.3 Medição de malha

A função `mesh_stats` é o instrumento de verificação estrutural. Ela conta faces por número de vértices e percorre as arestas de uma cópia `bmesh` para classificar cada uma pelo número de faces incidentes.

```
1 def mesh_stats(obj, evaluated: bool = False) -> dict:
2     me = obj.data
3     ngons = sum(1 for p in me.polygons if len(p.vertices) > 4)
4     tris = sum(1 for p in me.polygons if len(p.vertices) == 3)
5     quads = sum(1 for p in me.polygons if len(p.vertices) == 4)
6     bm = bmesh.new()
7     bm.from_mesh(me)
8     non_manifold = sum(1 for e in bm.edges if len(e.link_faces) not in (1, 2))
9     boundary = sum(1 for e in bm.edges if len(e.link_faces) == 1)
10    bm.free()
11    return dict(verts=len(me.vertices), faces=len(me.polygons),
12                quads=quads, tris=tris, ngons=ngons,
13                non_manifold=non_manifold, boundary=boundary)
```

É essa função que produz a evidência numérica discutida no Capítulo 6 e reaproveitada pelo relatório de malha.

5.3 Materiais

Foram definidos treze materiais. O Quadro 5.3 apresenta os principais parâmetros. A lista completa, com os valores efetivamente atribuídos, está no Apêndice B.

Material	Estratégia
MAT_Blue_Metallic	Pintura multicamada: base azul profunda, flocos por ruído, verniz e profundidade por peso de camada
MAT_Glass_Polycarbonate	Transmissão total, rugosidade 0,02, índice de refração 1,58
MAT_Headlamp_Lens	Transmissão total, rugosidade 0,03, índice 1,52
MAT_Lens_Red / MAT_Lens_Amber	Lentes coloridas com transmissão e rugosidade baixa
MAT_Rubber_Tire	Preto, rugosidade 0,85 com ruído em relevo
MAT_Aluminum_Brushed	Metálico, rugosidade 0,32, anisotropia 0,7
MAT_Chrome_Trim / MAT_Badge_Chrome / MAT_Reflector_Chrome	Metálico polido com rugosidade entre 0,05 e 0,10
MAT_Black_Trim	Preto, rugosidade 0,55 com ruído leve
MAT_Grille_Mesh	Preto escuro, rugosidade alta
MAT_Interior_Fabric	Preto, rugosidade 0,90 com ruído

Fonte: elaborado pelo autor (2026).

Quadro 5.3: Materiais definidos e estratégia de sombreado

5.3.1 A pintura multicamada

O material da carroceria é construído em três eixos independentes, representados na Figura 5.1. A cor base é um azul profundo; os flocos metálicos são um ruído de alta frequência ligado à normal e à rugosidade; e o verniz é uma camada dielétrica configurada com peso 0,6, rugosidade 0,02 e índice de refração 1,55. A profundidade de pigmento é obtida por um nó de peso de camada cuja saída de *facing* alimenta uma rampa de cor, usada para misturar um azul mais escuro nas regiões de incidência rasante.

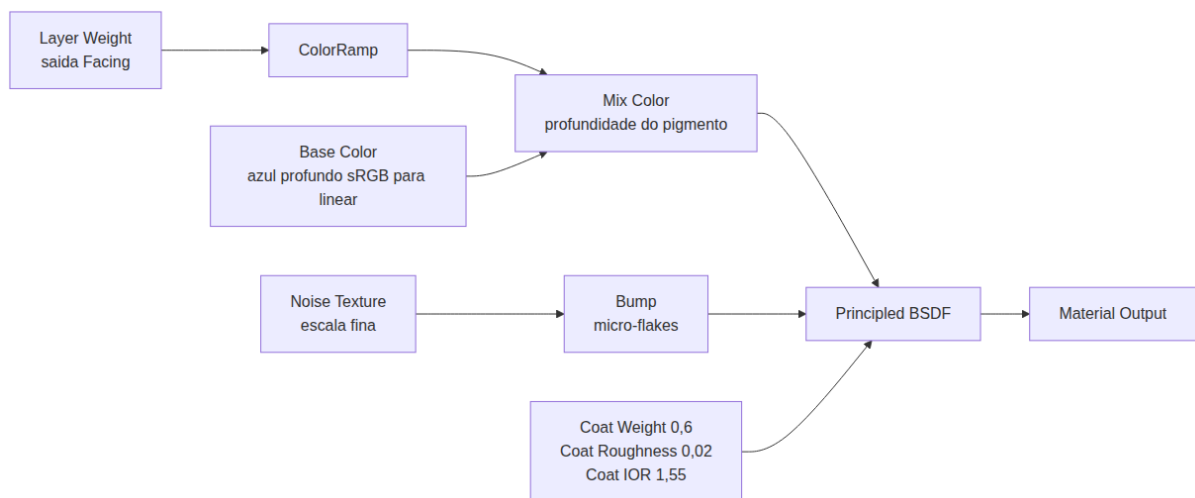


Figura 5.1: Estrutura do material de pintura. Os três caminhos — cor base, micro-relevo e verniz — são independentes e se combinam no nó de sombreado.

Fonte: elaborado pelo autor (2026).

5.3.2 Habilitação da refração

Uma armadilha específica do motor EEVEE merece registro. Um material com transmissão configurada *não* refrata automaticamente: é necessário habilitar a refração por traçado de raios tanto no material quanto na configuração de render. O código percorre os materiais transparentes e ativa as propriedades correspondentes, de forma tolerante à ausência do atributo em versões diferentes:

```
1 for key in ("glass", "headlamp_lens", "lens_red", "lens_amber"):
2     m = mats.get(key)
3     if m is None:
4         continue
5     m.use_screen_refraction = True
6     if hasattr(m, "use_raytrace_refraction"):
7         m.use_raytrace_refraction = True
8     m.blend_method = "BLEND" if hasattr(m, "blend_method") else m.blend_method
```

A tolerância por `hasattr` é uma decisão de engenharia defensiva: o script deve funcionar em versões próximas do Blender, degradando de forma controlada em vez de falhar.

5.4 O casco principal

5.4.1 Perfil transversal

O perfil de meio-lado tem nove pontos, definidos em função de sete parâmetros: a meia-largura máxima w , a meia-largura do topo w_t , a cota do fundo z_b , a cota do ombro z_s , a cota da aresta superior z_{te} e a cota do centro do topo z_t . A função espelha o meio-perfil e fecha o laço, resultando em dezesseis pontos por anel.

```
1 def body_ring(w, wt, zb, zs, zte, zt) -> list[tuple[float, float]]:
2     """Half profile mirrored - 16 points, closed loop."""
3     span = zs - zb
4     zm = zb + 0.58 * span
5     half = [
6         (0.00, zb),
7         (0.40 * w, zb + 0.02 * span),
8         (0.80 * w, zb + 0.22 * span),
9         (0.97 * w, zb + 0.46 * span),
10        (1.00 * w, zm),
11        (0.96 * w, zs),
12        (0.80 * w, zte - 0.030 * (zt - zte)),
13        (0.42 * wt, zte),
14        (0.00, zt),
15    ]
```

```
16 return half + [(-y, z) for (y, z) in reversed(half[1:-1])]
```

A ordem dos pontos é significativa: ela define a direção das cadeias de arestas que percorrem o casco no sentido longitudinal e, portanto, o fluxo de arestas do modelo. Os pontos 4 e 5 concentram a inflexão de largura máxima e o ombro, que são as regiões de maior curvatura e as que mais influenciam a leitura da silhueta.

5.4.2 Estações do casco

O casco é definido por **47 estações** transversais, cada uma com sua séxtupla $(x, w, w_t, z_s, z_{te}, z_t)$; a cota do fundo é calculada, e não tabelada, porque depende dos vãos de roda. A Tabela 5.1 mostra uma amostra; a tabela completa está no Apêndice A.

Tabela 5.1: Amostra de estações do casco (valores em metros)

x	w	w_t	z_s	z_{te}	z_t
1,863	0,625	0,500	0,470	0,560	0,592
1,740	0,792	0,646	0,530	0,618	0,650
1,545	0,808	0,660	0,570	0,658	0,690
1,150	0,820	0,660	0,680	0,742	0,766
0,700	0,792	0,630	0,592	0,700	0,742
0,020	0,797	0,626	0,640	0,712	0,748
-0,520	0,832	0,648	0,692	0,812	0,872
-1,150	0,859	0,657	0,740	0,874	0,911
-1,852	0,735	0,520	0,606	0,706	0,732
-1,858	0,735	0,520	0,606	0,706	0,732

Fonte: elaborado pelo autor a partir de *BODY_STATIONS* (2026).

Três características da tabela merecem leitura. Primeiro, a largura máxima ocorre próximo ao eixo traseiro ($w = 0,859$ m em $x = -1,150$), o que reproduz a distribuição de massa visual do veículo real, com a traseira mais larga que a dianteira. Segundo, a cota do topo cresce de 0,592 m no nariz até 0,911 m no deck, evidenciando a inclinação ascendente da superfície — a *waistline* referida na especificação. Terceiro, há estações repetidas no nariz (1,863 e 1,856) e na cauda (-1,852 e -1,858): são anéis de retenção, discutidos adiante.

O encadeamento é feito por uma função que cria os anéis e os conecta por quads, com atribuição opcional de material por faixa de faces:

```
1 def build_loft(name, stations, mat_grid=None, cap_mats=(0, 0),
2               materials=None, tag_info=None, subsurf=(1, 3),
3               smooth=True, caps=True):
4     bm = bmesh.new()
5     rings = []
6     for x, ring in stations:
7         rings.append([bm.verts.new((x, y, z)) for (y, z) in ring])
8     bm.verts.ensure_lookup_table()
9     nseg = len(rings[0])
```

```

10 bands = []
11 for i in range(len(rings) - 1):
12     row = []
13     for j in range(nseg):
14         k = (j + 1) % nseg
15         face = bm.faces.new((rings[i][j], rings[i][k],
16                               rings[i + 1][k], rings[i + 1][j]))
17         row.append(face)
18     bands.append(row)
19 if mat_grid is not None:
20     for i, row in enumerate(bands):
21         for j, face in enumerate(row):
22             face.material_index = mat_grid[i][j]
23 if caps:
24     for end, ring_index in ((0, 0), (1, len(rings) - 1)):
25         _folded_fan_cap(bm, list(rings[ring_index]), cap_mats[end], smooth)
26 ...

```

Como cada face conecta exatamente quatro vértices, a malha resultante é quadrangular por construção. Não existe etapa de correção topológica posterior: a qualidade é uma propriedade do algoritmo.

5.4.3 Arcos de roda sem booleanas

O recorte do arco é obtido por cálculo, e não por subtração. Para cada estação x , a cota do fundo é a maior entre a cota base da carroceria e a circunferência de cada arco:

```

1 def arch_bottom(x, base, arches) -> float:
2     """Body bottom accounting for wheel arches."""
3     z = base
4     for x_axle, z_axle, radius in arches:
5         dx = x - x_axle
6         if abs(dx) < radius:
7             z = max(z, z_axle + math.sqrt(radius * radius - dx * dx))
8     return z

```

Os parâmetros são $(x_{\text{eixo}}, z_{\text{eixo}}, r)$ para cada arco: dianteiro com raio 0,335 m e traseiro com raio 0,350 m, ambos centrados na cota do raio do pneu correspondente. A elevação do fundo nas estações próximas ao eixo produz o “túnel” do arco sem qualquer alteração na conectividade da malha, o que preserva a grade quadrangular e a continuidade dos reflexos sobre o para-lama.

5.4.4 Tampas de extremidade

Fechar o nariz e a cauda do *loft* é o ponto em que a maioria das implementações ingênuas introduz n-gons: o contorno tem dezesseis vértices, e preenchê-lo com uma única face produziria um n-gon de dezesseis lados. A biblioteca implementa uma tampa por “leque dobrado”, na qual cada quad conecta um par de vértices consecutivos ao par espelhado do lado oposto do laço:

```
1 def _folded_fan_cap(bm, verts, mat_index, smooth=True):
2     """Cap a closed loop using quads only (even n)."""
3     n = len(verts)
4     if n < 4:
5         return []
6     if n % 2:
7         n -= 1
8     created = []
9     for i in range(n // 2 - 1):
10         a, b = verts[i], verts[i + 1]
11         c, d = verts[n - 2 - i], verts[n - 1 - i]
12         if len({a, b, c, d}) < 4:
13             continue
14         try:
15             face = bm.faces.new((a, b, c, d))
16         except ValueError:
17             continue
18         face.material_index = mat_index
19         created.append(face)
20     return created
```

A técnica tem uma consequência geométrica que precisa ser compreendida: a tampa resultante não é plana, e sim uma superfície dobrada. Como a região é levemente curva e fica nas extremidades do veículo, o resultado é visualmente adequado e, sobretudo, topologicamente limpo. A alternativa — aceitar um n-gon e triangular depois — seria incompatível com o requisito de malha.

A mesma função é reutilizada para eliminar tampas n-gon geradas pelas primitivas de cilindro, o que unifica a garantia: qualquer face com mais de quatro lados é substituída por quads antes de a malha ser convertida em objeto.

5.4.5 Anéis de retenção

As estações repetidas no nariz e na cauda são um recurso deliberado. Em subdivisão, uma sequência de estações muito espaçadas nas extremidades faz com que a superfície limite retraia e arredonde o contorno — o resultado é o aspecto de “balão” mencionado na análise crítica do projeto. Inserir duas estações quase coincidentes (1,863 e 1,856; −1,852 e

−1,858) cria uma descontinuidade de segunda ordem que “segura” o contorno, achatando a face terminal e estabilizando a silhueta. O custo é de três faces por extremidade — desprezível.

5.4.6 Rebaixos reais

Dois recortes são feitos por deslocamento direto de vértices, com função de atenuação suave: as entradas de ar laterais, atrás das portas, e o alojamento dos faróis. O primeiro opera apenas nos vértices com $|y| > 0,35$ m e desloca-os para dentro segundo uma elipse com profundidade máxima de 62 mm:

```
1 def _scoop_dent(obj) -> None:
2     """Real side air-intake opening (no boolean, no normal map)."""
3     cx, cz = -0.235, 0.520
4     rx, rz = 0.215, 0.165
5     depth = 0.062
6     for v in obj.data.vertices:
7         if abs(v.co.y) < 0.35:
8             continue
9         nx = (v.co.x - cx) / rx
10        nz = (v.co.z - cz) / rz
11        r = math.sqrt(nx * nx + nz * nz)
12        if r >= 1.0:
13            continue
14        fade = 1.0 if r <= 0.62 else 1.0 - ((r - 0.62) / 0.38) ** 1.5
15        sign = 1.0 if v.co.y > 0 else -1.0
16        v.co.y -= sign * depth * fade
```

O expoente 1,5 na função de atenuação foi escolhido para produzir uma transição ligeiramente mais rápida que a quadrática, evitando uma borda excessivamente mole. A profundidade de 62 mm supera o mínimo de 30 mm exigido pelo requisito FR-006.

O alojamento dos faróis usa a mesma estratégia com uma atenuação cônica suave, escavando 30 mm no *clamshell* dianteiro em torno de duas elipses centradas em $y = \pm 0,470$ m.

5.5 Cabine, vidros e interior

A cabine é um segundo *loft*, assentado sobre a linha de cintura. Seu perfil tem dez pontos, com topo plano e laterais quase verticais, o que corresponde à geometria de um *hardtop* rígido:

```
1 def cabin_ring(wb, wt, zc, h) -> list[tuple[float, float]]:
2     """Cabin profile - flat top, near-vertical sides (10 points)."""
3     half = [
```

```

4         (wb, zc),
5         (0.99 * wb, zc + 0.30 * h),
6         (0.97 * wb, zc + 0.62 * h),
7         (0.90 * wb, zc + 0.88 * h),
8         (0.66 * wt, zc + 0.985 * h),
9         (0.00, zc + h),
10    ]
11    return half + [(-y, z) for (y, z) in reversed(half[1:-1])]

```

O vidro não é um objeto separado: ele é atribuído por índice de face, a partir de uma matriz de material que mapeia cada combinação de estação e segmento do perfil para um dos dois materiais da cabine. As zonas são declaradas explicitamente — para-brisa, teto, janela traseira e vidros laterais — e o mapeamento converte a intenção de projeto em índices:

```

1  # zones: 0..4 windshield, 5..7 roof, 8..10 rear window, 11+ buttress
2  windshield = set(range(0, 5))
3  roof = set(range(5, 8))
4  rear = {8, 9, 10}
5  side_glass = {5, 6, 7, 8}
6  top_segments = {3, 4, 5, 6}
7  side_segments = {0, 1, 7, 8}
8  mat_grid = []
9  for band in range(len(stations) - 1):
10     row = []
11     for seg in range(10):
12         if band in windshield and seg in top_segments:
13             row.append(1)
14         elif band in rear and seg in top_segments:
15             row.append(1)
16         elif band in side_glass and seg in side_segments:
17             row.append(1)
18         else:
19             row.append(0)
20     mat_grid.append(row)

```

A vantagem dessa abordagem é que não existem “placas” de vidro flutuando sobre a carroceria: a superfície do vidro é contínua com a do teto, e a transição entre material pintado e vidro ocorre exatamente onde a geometria muda. A desvantagem é o acoplamento entre a definição da malha e a definição do material: alterar o número de estações exige revisar as zonas.

O interior é simplificado, conforme o escopo: dois assentos (assento e encosto), painel, volante e santo-antónio, dimensionados para serem lidos através dos vidros sem pretender fidelidade de acabamento interno.

5.6 Rodas

Cada roda é composta por onze objetos: pneu, aro externo, disco do aro, cubo, tampa central, seis raios, disco de freio e pinça. O pneu é um sólido de revolução de um perfil fechado; o aro resulta da revolução de um perfil em três segmentos; e os raios são caixas chanfradas posicionadas por rotação em torno do eixo.

As dimensões vêm da designação comercial dos pneus:

```
1 TIRE_F_R, RIM_F_R, TIRE_F_HW = 0.292, 0.1905, 0.0925 # 185/55R15
2 TIRE_R_R, RIM_R_R, TIRE_R_HW = 0.306, 0.2032, 0.1025 # 205/50R16
3 ARCH_F = (AXLE_F_X, TIRE_F_R, 0.335)
4 ARCH_R = (AXLE_R_X, TIRE_R_R, 0.350)
```

A escolha de derivar o raio do pneu da própria designação — em vez de estimá-lo da fotografia — reduz uma fonte de erro: a geometria do pneu é conhecida, e o que a fotografia precisa informar é apenas a posição relativa do eixo e a altura da carroceria.

5.7 Conjuntos ópticos

O farol dianteiro é composto por cinco objetos: carcaça, refletor, dois elementos ópticos internos e a lente. A carcaça e o refletor são elipsoides achatados; a lente é uma casca de mesmo contorno, com o material de transmissão. O conjunto é assentado dentro do rebaixo escavado no *clamshell*, de modo que a lente fica aproximadamente no plano da superfície nominal.

A traseira tem quatro lanternas redondas, cada uma com lente e aro metálico, mais dois refletores no difusor. A posição das lanternas foi um ponto de correção significativo: como a superfície pós-subdivisão recua em relação à tampa nominal do *loft*, as lanternas inicialmente ficaram ocultas pela carroceria. A correção consistiu em deslocá-las para fora do envelope nominal ($\leq 1,863\text{ m}$), o que exemplifica um modo de falha próprio de projetos por subdivisão: a geometria de controle não é a geometria visível.

5.8 Ventilações e aerodinâmica

Compõem este grupo a grelha frontal, os *louvers* do *engine cover*, as saídas de ventilação traseiras, o difusor, as duas saídas de escape centrais, o *splitter* dianteiro e as aletas do difusor. A grelha frontal é um painel rebaixado com cinco aletas; os *louvers* são doze lâminas distribuídas em duas fileiras, cada uma uma caixa chanfrada inclinada.

Uma decisão de correção merece registro: a grelha e as inserções ópticas são objetos *assentados* sobre a superfície, e não recortes perfurados. A razão é geométrica: a tampa do *loft* não possui vértices internos que permitam escavar uma abertura sem recorrer a uma

operação booleana — que o requisito proíbe. Essa escolha é registrada como desvio de especificação, e não como atendimento pleno do requisito.

5.9 Emblemas

Os emblemas LOTUS e 111S são textos tridimensionais convertidos em malha. O processo cria um objeto de texto, define corpo, tamanho, extrusão, alinhamento e espaçamento entre caracteres, e converte a curva resultante em malha. O parâmetro de espaçamento foi necessário porque o *tracking* padrão da fonte deixava os caracteres excessivamente separados em relação à referência.

```
1 def text_mesh(name, body, size, extrude, location=(0, 0, 0),
2             rotation=(0, 0, 0), material=None, tag_info=None,
3             align_x="CENTER", align_y="CENTER", spacing=1.0):
4     curve = bpy.data.curves.new(name=name, type="FONT")
5     curve.body = body
6     curve.size = size
7     curve.extrude = extrude
8     curve.align_x = align_x
9     curve.align_y = align_y
10    curve.space_character = spacing
11    obj = bpy.data.objects.new(name, curve)
12    bpy.context.scene.collection.objects.link(obj)
13    bpy.context.view_layer.objects.active = obj
14    bpy.ops.object.convert(target="MESH")
15    ...
```

Os glifos têm contorno aberto, o que gera arestas de borda na malha resultante. Essas arestas não são variedade — portanto não violam o requisito — mas são a *única* origem das 1.776 arestas de borda registradas no relatório. A correção indicada é substituir o texto por glifos de contorno fechado, extrusados; a solução não foi aplicada nesta versão e está registrada como pendência.

5.10 Nomenclatura, metadados e subdivisão

Todo objeto recebe um prefixo de categoria: GEO_ para geometria, LGT_ para luzes e CAM_ para câmeras. Além do nome, cada objeto de geometria recebe três propriedades personalizadas, gravadas pela função `tag`:

```
1 def tag(obj, cat: str, part: str, swap: str) -> None:
2     """Metadata for automated material swap via MCP."""
3     obj["cat"] = cat
4     obj["part"] = part
5     obj["mat_swap_key"] = swap
```

O objetivo declarado é permitir troca automatizada de material: um agente pode localizar todos os objetos cujo `mat_swap_key` seja `paint_primary` e substituir o material sem inspecionar nomes de objeto. É um exemplo de como a convenção de nomenclatura deixa de ser estética e passa a ser uma interface de programação.

Os dois corpos principais recebem o modificador de subdivisão com nível de visualização 1 e nível de render 3. A assimetria é deliberada: o nível 3 no render garante a suavidade do resultado final, enquanto o nível 1 na visualização mantém a resposta da interface utilizável — requisito prático para quem precisa inspecionar a cena.

5.11 Relatório de malha

O *build* termina emitindo um relatório em formato de texto, com uma linha por objeto e totais consolidados. O relatório é a evidência primária de integridade estrutural, e contém: contagem de vértices, faces, quads, triângulos e n-gons por objeto; arestas não variedade; arestas de borda; e o envelope dimensional com a diferença em relação aos alvos.

O envelope é calculado varrendo os oito cantos da caixa delimitadora de cada objeto de malha, transformados para coordenadas de mundo:

```
1 def world_bounds(objects=None) -> dict:
2     objs = objects if objects is not None else bpy.data.objects
3     lo = Vector((1e9, 1e9, 1e9))
4     hi = Vector((-1e9, -1e9, -1e9))
5     for obj in objs:
6         if obj.type != "MESH":
7             continue
8         for corner in obj.bound_box:
9             p = obj.matrix_world @ Vector(corner)
10            lo = Vector((min(lo.x, p.x), min(lo.y, p.y), min(lo.z, p.z)))
11            hi = Vector((max(hi.x, p.x), max(hi.y, p.y), max(hi.z, p.z)))
12     return dict(min=lo, max=hi, size=hi - lo)
```

A escolha de medir a caixa delimitadora dos objetos, e não os vértices da malha de controle, é relevante: é a geometria avaliada que define o envelope percebido. O relatório é gravado em `docs/renders/mesh_report.txt` e reproduzido no Apêndice C.

5.12 Rig de verificação e câmeras

O rig de verificação tem quatro luzes de área retangulares: uma chave, um preenchimento, uma luz de recorte e um cartão longo lateral. As quatro são direcionadas para um ponto comum por uma função auxiliar que converte a direção em rotação de rastreamento:

```

1 def _aim(obj, target=(0.0, 0.0, 0.55)) -> None:
2     direction = Vector(target) - obj.location
3     obj.rotation_euler = direction.to_track_quat("-Z", "Y").to_euler()

```

As nove câmeras formam um conjunto de verificação e de apresentação. A Quadro 5.4 lista as câmeras com distância focal e posição. A vista superior usa rotação explícita em vez de rastreamento, porque o rastreamento com alvo diretamente abaixo produz uma rotação degenerada.

Câmera	Posição (m)	Focal	Papel
CAM_Frontal	(7,6; 0; 0,72)	95 mm	vista frontal
CAM_Lateral	(0; 8,2; 0,76)	62 mm	vista lateral
CAM_Traseira	(-7,6; 0; 0,78)	95 mm	vista traseira
CAM_Superior	(0; 0; 10,5)	50 mm	vista superior
CAM_WheelDetail	(2,05; 1,35; 0,50)	78 mm	detalhe de roda
CAM_TresQuartos	(4,95; 4,15; 1,95)	72 mm	três-quartos
CAM_TresQuartos_Tr	(-4,95; 4,15; 1,95)	72 mm	três-quartos traseiro
CAM_Isometrica	(5,40; 5,40; 3,90)	70 mm	isométrica
CAM_Hero	(4,90; 3,80; 1,00)	50 mm	apresentação

Fonte: elaborado pelo autor a partir de `build_cameras` (2026).

Quadro 5.4: Câmeras do conjunto e parâmetros principais

5.13 Configuração de render e mudanças de API

A configuração de render é feita por código e precisa acomodar mudanças introduzidas nas versões recentes do Blender. Três delas afetam diretamente este projeto e são tratadas defensivamente.

Ausência do árvore de composição na cena. Nas versões anteriores, o compositor era acessado por uma propriedade da cena. Na versão utilizada, o compositor é um grupo de nós criado separadamente e associado à cena, e o nó de saída `Composite` foi substituído por um nó de saída de grupo.

Ausência do efeito de brilho no motor. O parâmetro de *bloom* deixou de existir; o brilho passou a ser obtido por um nó de *glare* no compositor, com tipo, limiar, tamanho e intensidade configurados como entradas do nó.

Nomes de operações de cor. Os nomes válidos do *look* de mapeamento de tons variam entre versões, e um nome inválido interrompe a execução. A solução adotada é uma tentativa em cascata:

```

1 scene.view_settings.view_transform = "AgX"
2 for look in ("AgX - Medium High Contrast", "AgX - Base Contrast", "None"):
3     try:
4         scene.view_settings.look = look
5         break

```

```

6     except Exception:
7         continue
8 scene.view_settings.exposure = -0.18
9 for attr, value in (("taa_render_samples", 96),
10                    ("use_raytracing", True),
11                    ("use_shadows", True)):
12     if hasattr(scene.eevee, attr):
13         try:
14             setattr(scene.eevee, attr, value)
15         except Exception:
16             pass

```

O padrão é sempre o mesmo: verificar a existência do atributo antes de usá-lo, e falhar de forma tolerante. Em um contexto de automação por agente, essa disciplina tem valor adicional: uma exceção não tratada interrompe a sessão e obriga a um novo ciclo de diagnóstico, enquanto uma degradação controlada mantém o pipeline executável.

Uma armadilha adicional, de natureza diferente, foi verificada: a atribuição de escala a um objeto exige uma tupla de três elementos. Construções que produzem tuplas de tamanho variável — como a repetição de um valor — não são aceitas nessa atribuição. É um exemplo de erro que só se manifesta em tempo de execução dentro do Blender, e que reforça a necessidade de validação sintática prévia do código gerado.

5.14 A galeria de apresentação

A galeria produz cinco imagens: superior, traseira, lateral, isométrica e uma imagem com efeito de luz. As quatro primeiras usam o rig neutro. A quinta usa um rig dramático temporário, com luzes de recorte laterais, emissão nos elementos ópticos dianteiros e brilho no compositor. Ao final, o script restaura o rig neutro e *não* grava o arquivo de cena.

A decisão de não persistir o rig dramático é deliberada: ele é um artefato de apresentação, e não um estado do modelo. Persisti-lo contaminaria o ambiente de verificação e quebraria a reprodutibilidade dos renders neutros. A Figura 5.2 mostra o resultado do rig dramático.

5.15 O ciclo de chamadas via MCP

A operação por MCP reduz o ciclo de trabalho a três ferramentas: executar código, consultar a cena e capturar a imagem da *viewport*. O ciclo completo de uma iteração é apresentado na Figura 5.3.

O encadeamento tem uma propriedade importante: a mensagem de execução é, ela própria, o artefato de reprodutibilidade. Executar o mesmo script por linha de comando ou por MCP produz a mesma cena, porque nos dois casos o que se executa é o mesmo

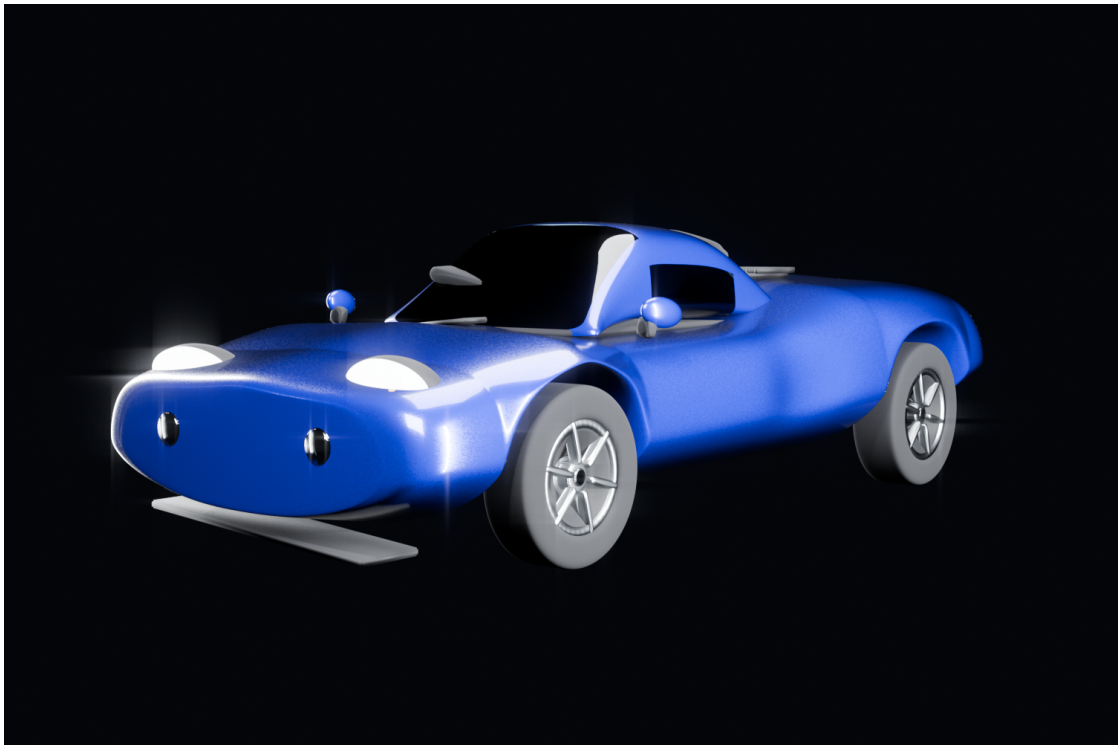


Figura 5.2: Imagem de apresentação com rig dramático temporário: luzes de recorte laterais, elementos ópticos acesos e brilho via nó de *glare*.

Fonte: elaborado pelo autor (2026).

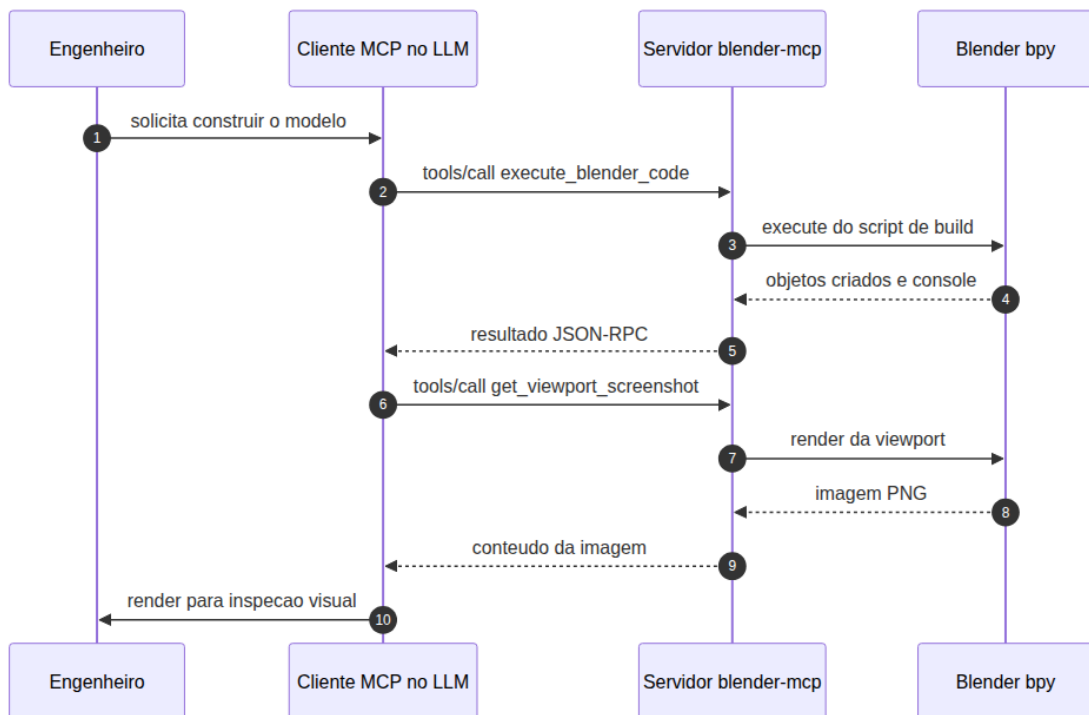


Figura 5.3: Ciclo de chamadas do protocolo durante um ciclo de construção e inspeção. Cada chamada é uma mensagem serializada, o que constitui o registro auditável do processo.

Fonte: elaborado pelo autor (2026).

código. A diferença entre os dois modos está no *registro*: no modo MCP, os argumentos e resultados ficam serializados; no modo linha de comando, restam no histórico do terminal.

Uma limitação merece ser dita com clareza. A inspeção visual mediada por MCP consiste em uma captura bidimensional. A avaliação de continuidade de superfície por essa via é inferior à inspeção interativa, na qual o avaliador pode orbitar a cena e observar o comportamento especular sob movimento. O presente trabalho compensa essa limitação com duas providências: a captura de vistas múltiplas predefinidas e a verificação numérica das propriedades estruturais. Ainda assim, a limitação permanece e é registrada como tal.

6 RESULTADOS

Este capítulo apresenta exclusivamente o que foi obtido: números extraídos do relatório de malha, tempos registrados na sessão de referência, contagens de objetos e análise comparativa das vistas.

6.1 Protocolo executado e tempos

O build foi executado dentro de uma sessão viva do Blender, mediada por MCP. A Tabela 6.1 registra os tempos dos renders da etapa de verificação, executados a 1500×1000 pixels com 64 amostras de acumulação temporal.

Tabela 6.1: Tempos de render da etapa de verificação (EVEE, 64 amostras, 1500×1000)

Vista	Arquivo	Tempo (s)
Frontal	01_frontal.png	6,3
Superior	02_superior.png	3,9
Lateral	03_lateral.png	4,0
Traseira	04_traseira.png	4,8
Três-quartos	05_three_quarter.png	5,9
Três-quartos traseiro	06_three_quarter_rear.png	5,8
Detalhe de roda	07_wheel_closeup.png	6,4
Lateral em material neutro	10_clay_lateral.png	3,6
Três-quartos em material neutro	11_clay_three_quarter.png	4,8
Total das nove vistas	—	45,4

Fonte: registro da sessão de referência (SUMMARY.md); a soma aritmética das parcelas arredondadas é 45,5 s.

O tempo de construção completo da cena é da ordem de 0,6 s — cerca de setenta e cinco vezes menor que o tempo total de render. Essa assimetria tem consequência prática para o método: é barato reconstruir a geometria e caro verificá-la, o que justifica investir na verificabilidade automática das propriedades estruturais. A galeria de apresentação, executada a 96 amostras, consumiu aproximadamente 37 s para as cinco imagens.

6.2 Resultado do build

A Tabela 6.2 consolida os números do último build. Todos os valores são reproduzíveis por reexecução do script e estão registrados em `docs/renders/mesh_report.txt`.

Dois números merecem leitura atenta. O primeiro é a contagem de n-gons: zero. Trata-se de consequência direta das decisões de arquitetura — tampa por leque dobrado

Tabela 6.2: Resumo numérico do build

Grandeza	Valor
Objetos de malha	130
Materiais	13
Luzes	4
Câmeras	9
Faces (malha base)	32.632
quads	29.272
triângulos	3.360
n-gons	0
Arestas não variedade	0
Arestas de borda	1.776

Fonte: elaborado pelo autor a partir de docs/renders/mesh_report.txt (2026).

(Seção 5.4.4) e substituição sistemática de tampas n-gon —, e não de uma etapa de limpeza. O segundo é a contagem de arestas de borda: 1.776, todas concentradas nos três objetos de emblema, cujos glifos têm contorno aberto. Nenhum corpo curvo possui aresta de borda.

Objeto	Tris	Origem
GEO_Badge_LOTUS	604	glifos de texto com contorno aberto
GEO_Badge_111S	252	glifos de texto com contorno aberto
GEO_Badge_Front	48	canto de chanfro em caixa
Demais 127 objetos	2.456	cantos de chanfro de 3 segmentos em primitivas
Total	3.360	nenhum em superfície curva principal

Fonte: elaborado pelo autor a partir de mesh_report.txt (2026).

Quadro 6.1: Origem dos triângulos e das arestas de borda

6.3 Integridade topológica dos corpos principais

Os dois corpos que definem a superfície visível apresentam topologia integralmente quadrângular. O casco tem 750 quads, 752 vértices, zero n-gons, zero arestas não variedade e zero arestas de borda. A cabine tem 148 quads em 150 vértices, também sem n-gons, sem não variedade e sem borda.

Objeto	Vértices	Faces	Quads	N-gons
GEO_Body_Hull	752	750	750	0
GEO_Cabin_Hull	150	148	148	0
GEO_RollBar	896	896	896	0
GEO_Steering_Wheel	640	640	640	0
GEO_Wheel_*_Tire (4)	2.880	2.880	2.880	0

Fonte: elaborado pelo autor a partir de mesh_report.txt (2026).

Quadro 6.2: Composição das malhas principais

A verificação aritmética confirma a construção: 47 estações geram 46 faixas, cada uma com 16 faces, totalizando 736 quads; as duas tampas por leque dobrado acrescentam

7 quads cada, alcançando 750 — exatamente o valor medido. Do mesmo modo, a cabine: 15 estações geram 14 faixas de 10 faces (140 quads) e duas tampas de 4 quads cada, totalizando 148. A coincidência entre o cálculo e a medição é a evidência de que a malha é exatamente a que o algoritmo descreve.

6.4 Envelope dimensional

A Tabela 6.3 confronta os valores medidos com os alvos. As tolerâncias adotadas são de ± 2 cm em cada dimensão.

Tabela 6.3: Envelope dimensional: alvo, valor medido e desvio

Dimensão	Alvo (m)	Medido (m)	Desvio (cm)	Situação
Comprimento (X)	3,726	3,736	+1,0	dentro da tolerância
Largura (Y)	1,718	1,718	0,0	exato
Altura (Z)	1,117	1,117	0,0	exato
Solo (Z mínimo)	0,000	0,0000	—	exato
Centro em Y	0,000	0,0000	—	exato

Fonte: elaborado pelo autor a partir de `mesh_report.txt` (2026).

O único desvio é de +1,0 cm no comprimento, atribuível às tampas de extremidade: a tampa por leque dobrado é uma superfície não plana que avança ligeiramente além do plano da última estação. O valor permanece dentro da tolerância de ± 2 cm. A largura e a altura são exatas porque a largura máxima e a cota do topo são parâmetros diretos das estações, e não grandezas emergentes.

6.5 Conteúdo da cena

A Tabela 6.4 distribui os 130 objetos de malha por grupo funcional. A distribuição é informativa sobre o esforço de modelagem: as rodas, sozinhas, respondem por quarenta por cento dos objetos da cena, o que reflete seu número de partes reais (aro, disco, cubo, tampa, seis raios, pneu, disco de freio e pinça).

6.6 Análise visual comparativa

A análise segue a ordem das vistas. Cada figura justapõe a referência e o render correspondente, na mesma escala relativa.

Tabela 6.4: Objetos de malha por grupo funcional

Grupo	Objetos	Exemplos
Rodas	52	GEO_Wheel_FL_Tire, _Spoke_1
Ventilações	23	GEO_EngineCover_Louver_L_1
Interior	7	GEO_Steering_Wheel, GEO_RollBar
Lanternas traseiras	8	GEO_TailLamp_Lens_L_1
Grelha frontal	6	GEO_Grille_Bar_1, GEO_Grille_Recess
Aerodinâmica	6	GEO_Rear_Diffuser, GEO_Front_Splitter
Faróis e indicadores	12	GEO_Headlamp_Lens_L
Faróis auxiliares	4	GEO_DrivingLamp_L_1
Espelhos	4	GEO_Mirror_Shell_L
Casco e cabine	3	GEO_Body_Hull, GEO_Cabin_Hull
Emblemas	3	GEO_Badge_LOTUS, GEO_Badge_111S
Refletores do difusor	2	GEO_Diffuser_Reflector_L
Total	130	—

Fonte: elaborado pelo autor a partir de mesh_report.txt (2026).

6.6.1 Vista frontal

A Figura 6.1 confronta o render frontal com a referência. Os dois faróis ovais, a grelha central rebaixada e os dois faróis auxiliares redondos coincidem em posição relativa. A linha do capô apresenta a inflexão descendente característica sobre cada para-lama. O critério CA-005 é atendido.



Figura 6.1: Vista frontal: referência (esquerda) e render (direita).

Fonte: acervo do projeto e render próprio (2026).

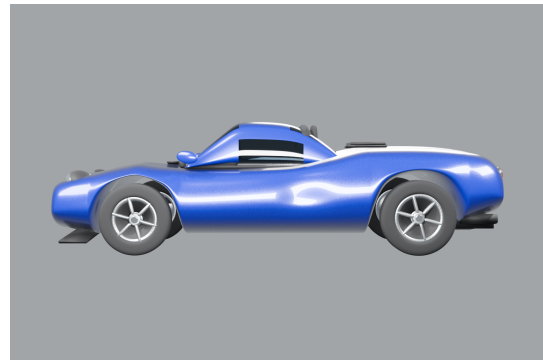
6.6.2 Vista lateral

A Figura 6.2 é a vista mais informativa e a que expõe a limitação mais relevante do resultado. As proporções gerais — altura da cintura, posição dos eixos, relação entre o comprimento do capô e o da traseira, arco de roda proeminente — são coerentes com a referência. O nariz baixo e a cauda truncada estão presentes.

O desvio identificado é de natureza diferente: falta à superfície a *linha de caráter* lateral, a curva de raio pequeno que atravessa a lateral do veículo e que, na referência, produz uma quebra deliberada nos reflexos ao longo de toda a extensão. No modelo, essa região é suave, porque a subdivisão suaviza qualquer variação que não esteja sustentada por um anel de proximidade. O resultado é um corpo ligeiramente mais arredondado que o real. Por isso, CA-004 é classificado como **parcial**.



referência



render

Figura 6.2: Vista lateral: referência (esquerda) e render (direita). A silhueta geral coincide; a linha de caráter lateral está ausente.

Fonte: acervo do projeto e render próprio (2026).

6.6.3 Vista traseira

A Figura 6.3 mostra a traseira. Estão presentes as quatro lanternas redondas com aro metálico, os emblemas LOTUS e 111S como malha tridimensional, o difusor com duas saídas de escape centrais e os dois refletores. Os critérios CA-006 e CA-009 são atendidos.

Vale registrar um artefato observado: em ampliação, a superfície do texto LOTUS apresenta facetas nos cantos dos glifos, consequência dos triângulos de chanfro discutidos no Quadro 6.1. Na escala de observação normal da vista traseira, o efeito não é perceptível.



referência



render

Figura 6.3: Vista traseira: referência (esquerda) e render (direita).

Fonte: acervo do projeto e render próprio (2026).

6.6.4 Vista superior

A Figura 6.4 compara a vista superior com a referência de mesma orientação. A imagem revela as entradas de ar laterais atrás das portas, os *louvers* do *engine cover*, a posição relativa da cabine e o santo-antônio. O critério CA-008 é atendido.



referência



render

Figura 6.4: Vista superior: referência (esquerda) e render (direita).

Fonte: acervo do projeto e render próprio (2026).

6.6.5 Vista em três-quartos e pintura

A Figura 6.5 confronta o render em três-quartos com a referência azul, que é a imagem de mesma cor do material. Os reflexos são contínuos ao longo do para-lama dianteiro e da lateral, sem facetas ou ondulações visíveis — o que confirma a integridade da superfície. O tom, porém, ficou ligeiramente mais escuro que a referência. Por isso CA-007 é **parcial**.

Duas causas concorrem para o desvio de tom, e ambas já foram discutidas: a exposição global foi deliberadamente reduzida ($-0,18$) para preservar as altas luzes nos reflexos especulares; e a referência fotográfica passou pelo mapeamento de tons da câmera, diferente do AgX. Isolar a contribuição de cada causa exigiria um experimento controlado com um alvo de cor físico, que não foi realizado.

6.6.6 Detalhe da roda

A Figura 6.6 mostra o *close* da roda dianteira. Os seis raios estão presentes e distintos, com a tampa central, o disco e a pinça de freio. O requisito FR-004, que define a identidade da variante pelas rodas de seis raios, é atendido; o pneu apresenta o perfil e a proporção derivados da designação 185/55R15.

6.6.7 Vistas em material neutro

A Figura 6.7 apresenta as duas vistas em material neutro. Sua função é remover a variável da cor e do verniz e permitir a leitura da forma. A vista lateral em material



referência



render

Figura 6.5: Vista em três-quartos: referência (esquerda) e render (direita). Reflexos contínuos; tom ligeiramente mais escuro no render.

Fonte: acervo do projeto e render próprio (2026).



Figura 6.6: Detalhe da roda dianteira: seis raios, tampa central, disco e pinça.

Fonte: elaborado pelo autor (2026).

neutro é a evidência mais clara da limitação de silhueta: sem o verniz, a superfície é uniformemente suave, e a ausência da aresta viva ao longo do para-lama torna-se evidente, ao contrário do que ocorreria se o reflexo do verniz mascarasse a leitura.

6.7 Resultado por critério de aceitação

O Quadro 6.3 consolida o resultado. Sete dos nove critérios foram plenamente atendidos; dois ficaram parciais.

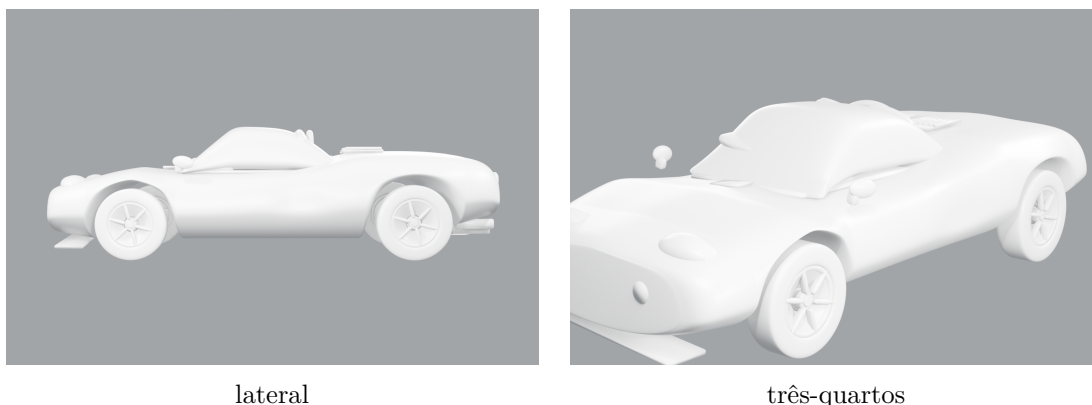


Figura 6.7: Vistas em material neutro, para leitura de forma sem influência de cor e verniz.

Fonte: elaborado pelo autor (2026).

ID	Critério	Status	Evidência
CA-001	Build sem erro	PASS	130 objetos GEO_*
CA-002	Envelope em ± 2 cm	PASS	+1,0 cm em X; Y e Z exatos
CA-003	Zero n-gons e normais coerentes	PASS	0 n-gons; 0 não variedade
CA-004	Silhueta lateral coerente	PARCIAL	proporções corretas; falta a linha de caráter
CA-005	Frente com faróis, grelha e auxiliares	PASS	Figura 6.1
CA-006	Traseira completa	PASS	Figura 6.3
CA-007	Pintura com reflexos contínuos	PARCIAL	reflexos contínuos; tom mais escuro
CA-008	Vista superior completa	PASS	Figura 6.4
CA-009	Nomenclatura e metadados	PASS	cat, part, mat_swap_key

Fonte: elaborado pelo autor (2026).

Quadro 6.3: Resultado da verificação por critério de aceitação

6.8 Síntese dos resultados

Os resultados sustentam quatro afirmações verificáveis. Primeira: a geometria é integralmente quadrangular nos corpos principais, com zero n-gons e zero arestas não variedade em toda a cena. Segunda: o envelope dimensional atende à tolerância, com desvio máximo de 1,0 cm. Terceira: os elementos de identidade da variante — rodas de seis raios, conjunto óptico, entradas de ar com geometria real, emblemas — estão presentes e são verificáveis nos renders. Quarta: a fidelidade de silhueta e de tom é parcial, e as causas são identificadas e localizadas.

A ausência de um controle experimental — um segundo modelo construído pelo método manual — impede que se afirme, com base em medição, que este pipeline seja *melhor* que o tradicional. O que se pode afirmar é que o artefato é reconstruível por um comando, que suas propriedades estruturais são medidas automaticamente e que cada decisão geométrica é rastreável até uma linha de código. A discussão dessas implicações é o objeto do capítulo seguinte.

7 DISCUSSÃO

Este capítulo interpreta os resultados, examina as causas dos critérios parciais, avalia o que a mediação por MCP efetivamente agrega, compara o trabalho com a literatura e explicita as limitações que permanecem.

7.1 Interpretação dos resultados

O resultado mais sólido do trabalho não é visual, e sim estrutural: zero n-gons e zero arestas não variedade em uma cena de 32.632 faces, com os dois corpos principais integralmente quadrangulares. A relevância disso está em que a propriedade foi obtida *por construção*, e não por limpeza posterior. Um modelo gerado por ferramentas interativas tipicamente requer uma etapa de retopologia — trabalho manual, difícil de reproduzir e de auditar. Aqui, a propriedade decorre do algoritmo de tampa e da estrutura de encadeamento, o que significa que ela se preserva automaticamente em qualquer reconstrução.

O segundo resultado relevante é a coincidência aritmética entre o previsto pela estrutura do código e o medido no relatório: 46 faixas de 16 faces mais duas tampas de 7 quads resultam em 750, exatamente o valor medido. Essa coincidência tem valor metodológico: ela mostra que o relatório mede o que o código descreve, e não um artefato de avaliação inserido *a posteriori* pela subdivisão.

O terceiro resultado é negativo e igualmente informativo: a inspeção visual *comparativa* identificou com clareza um desvio que a medição numérica não poderia detectar. Nenhum dos números do relatório — faces, n-gons, variedade, envelope — distingue um modelo com linha de caráter de um sem ela. A conclusão metodológica é que os dois tipos de verificação são complementares e não substituíveis, conforme argumentado no Capítulo 3.

7.2 Sobre os critérios parciais

7.2.1 A silhueta e o problema da aresta viva

O critério CA-004 ficou parcial porque falta a linha de caráter lateral — a curva de raio pequeno que percorre a lateral do veículo. A causa é estrutural e vale a pena explicitá-la, porque ela ilustra um limite do método de *loft*.

Um *loft* de estações transversais interpola suavemente ao longo do eixo longitudinal.

Se uma feição exige uma variação de raio pequeno, ela só sobrevive à subdivisão se estiver sustentada por anéis de proximidade (Capítulo 2). Sem eles, a subdivisão — que existe precisamente para suavizar — faz o que se espera dela e elimina a descontinuidade. O modelo, portanto, é vítima do seu próprio instrumento: a suavidade que garante a continuidade dos reflexos é a mesma que apaga a linha de caráter.

A correção é conhecida, e não requer mudança de método: inserir pares de estações próximas ao longo da linha pretendida, ou marcar a cadeia de arestas correspondente com peso de dobra. Os anéis de retenção já implementados no nariz e na cauda são a prova de conceito da técnica. O que faltou foi *aplicá-la* à lateral, e não saber como aplicá-la.

Há, portanto, uma lição sobre o próprio método de especificação. A especificação exigia “capturar a linha de cintura ascendente”, mas não especificava *como verificar* que a linha estava presente. Um critério de aceitação mais preciso — por exemplo, “a vista lateral em material neutro apresenta uma descontinuidade de curvatura visível ao longo de $x \in [-1, 0; +1, 0]$ ” — teria orientado a verificação desde o início. Critérios precisam ser *observáveis*, e não apenas corretos.

7.2.2 O tom da pintura

O critério CA-007 ficou parcial por um desvio de tom. A análise identificou três fatores concorrentes, mas não foi possível quantificar a contribuição de cada um.

O primeiro é a exposição global negativa $(-0, 18)$, adotada para preservar as altas luzes. Trata-se de uma decisão de compromisso: sem ela, os reflexos especulares no para-lama estouravam e mascaravam a leitura de continuidade. O efeito colateral é o escurecimento geral.

O segundo é a diferença de cadeia de imagem: a referência foi processada pela câmera e pelo *software* fotográfico; o render passou pelo AgX. Comparar tom entre duas cadeias diferentes sem um alvo de cor comum é, em rigor, uma comparação inválida.

O terceiro é a escolha do próprio valor de cor base. O azul foi definido como #0C1A96 convertido para linear, um valor escuro. Aumentar a luminosidade da cor base é a correção mais direta, mas só faz sentido depois de eliminar a contribuição de exposição e cadeia de imagem.

A lição metodológica é que um critério de aceitação sobre cor é, sem referência colorimétrica, um critério mal posto. Ele se aproxima mais de uma opinião fundamentada do que de uma verificação. Reconhecer isso é mais útil que tentar ajustar o tom por tentativa e erro até a coincidência visual.

7.3 O que a mediação por MCP agrega

O MCP foi empregado como meio, e não como objeto. Ainda assim, a análise do seu papel produz três conclusões.

A serialização é o ganho principal. O protocolo não torna o Blender mais capaz: todas as operações realizadas seriam possíveis por *script* executado na linha de comando. O que ele agrega é que cada interação é uma mensagem estruturada com nome de ferramenta e argumentos. Isso desloca o registro do processo de um histórico de terminal — que se perde, e que não é verificado por ninguém — para um conjunto de mensagens inspecionáveis.

A verificação visual por agente é limitada. A inspeção mediada por captura de *viewport* é estritamente inferior à inspeção interativa: o avaliador não pode orbitar, aproximar nem alterar a iluminação para testar hipóteses sobre a superfície. A limitação é intrínseca à natureza bidimensional da captura [35]. Neste trabalho, ela foi compensada por vistas múltiplas predefinidas e por verificação numérica, mas continua a existir.

O valor está na reprodutibilidade, não na velocidade. O tempo de build (0,6s) é irrelevante perto do tempo de render (45s para nove vistas). Não há, neste caso, ganho de velocidade apreciável em relação a um fluxo manual bem executado. O ganho é que o artefato pode ser reconstruído identicamente, revisado por diferença de código e verificado por medição automática. É um ganho de *qualidade de processo*, e não de produtividade bruta.

Uma consequência prática dessa análise é que o uso de agentes via MCP faz mais sentido em geometria *paramétrica e repetível* — rodas, louvres, grelhas, conjuntos de peças com variação — do que em superfícies nas quais o julgamento estético contínuo é dominante. A distribuição de objetos apresentada na Tabela 6.4 é sugestiva: cinquenta e dois objetos de roda foram gerados por uma única função parametrizada, enquanto a superfície do casco — um único objeto — exigiu a maior parte do esforço de iteração.

7.4 Comparação com a literatura

Frente à modelagem procedimental clássica [38, 39, 40], este trabalho não propõe gramática de forma nova. Sua contribuição, nessa frente, é mostrar que a parametrização por tabela de estações — um método simples e pouco citado — é suficiente para uma morfologia automotiva plausível quando combinada com subdivisão e controle de densidade.

Frente à engenharia reversa geométrica [42, 12], a diferença é de entrada e de objetivo. A literatura parte de medições densas e visa manufatura; aqui, parte-se de seis fotografias em perspectiva e visa renderização. Essa diferença desloca o problema de fidelidade micrométrica para fidelidade de morfologia e proporção — um regime em que a ausência de blueprints ortográficos é a principal fonte de erro, conforme discutido nas ameaças à validade.

Frente à reconstrução por imagens [43, 45, 46], a diferença é de natureza do artefato. Uma reconstrução por *Structure-from-Motion* ou por campos de radiância produziria uma representação visualmente mais fiel sob os pontos de vista observados, mas com malha

irregular e sem fluxo de arestas — inadequada ao requisito de topologia. É possível que a combinação dos dois métodos seja mais interessante que qualquer um isoladamente: usar reconstrução densa para *validar* proporções e o modelo procedimental para produzir a malha editável. Essa combinação não foi explorada aqui e é sugerida como trabalho futuro.

Frente aos agentes com ferramentas [8, 47, 10] e ao próprio MCP [2], a contribuição é um relatório de caso com verificação. A literatura descreve a capacidade de invocar ferramentas; este trabalho registra o que aconteceu quando essa capacidade foi aplicada a um problema com critério de aceitação externo — e registra também que o agente não substitui o julgamento estético nem a decisão de engenharia sobre o que verificar.

7.5 Limitações

Além das ameaças à validade já discutidas na Seção 4.13, quatro limitações são inseparáveis do resultado obtido.

Ausência de avaliação independente. A avaliação visual foi feita pelo autor do modelo. Sem avaliação cega e sem múltiplos avaliadores, não há como estimar a concordância do julgamento.

Ausência de referência colorimétrica. Como discutido, o critério de cor não é verificável sem um alvo físico e uma cadeia comum de captura.

Verificação de continuidade por inspeção dos reflexos. A especificação original pedia validação por mapas de zebra e isofotas. Essa validação foi declarada fora de escopo por decisão do usuário, o que significa que a continuidade G^2 alegada no material e na geometria não foi medida — apenas observada. Uma afirmativa forte como “continuidade G^2 ” exigiria análise dedicada.

Estudo de caso único. Há um único veículo, um único acabamento e uma única configuração de *software*. As conclusões gerais são hipóteses, e não resultados.

7.6 Implicações

Para a prática de modelagem, o trabalho sugere três diretrizes.

A primeira é *impor as restrições topológicas no algoritmo*. Verificar n-gons ao final do processo e corrigi-los é uma batalha perdida; construir unicamente estruturas que não os produzem é uma vitória permanente. A função de tampa por leque dobrado é um exemplo pequeno e reutilizável dessa postura.

A segunda é *separar a verificação por natureza da grandeza*. Grandezas estruturais são medidas; grandezas perceptuais são inspecionadas. Misturar as duas leva tanto a medir o que não importa quanto a aceitar o que não se pode medir.

A terceira é *tratar a automação por agente como infraestrutura, e não como substituto do julgamento*. O agente executa, mede e reporta com fidelidade notável; ele

não decide o que é bonito, nem que tolerância é aceitável, nem que desvio merece ser perseguido. Neste trabalho, a análise visual foi o instrumento decisivo para identificar o único desvio que a medição não alcançava.

7.7 Contribuições

Sintetizando, o trabalho contribui com: (i) um gerador procedimental completo e reproduzível de um veículo esportivo, com topologia controlada por construção; (ii) uma técnica de tampa de *loft* que produz apenas quads, aplicável a qualquer superfície fechada por anéis; (iii) uma demonstração de arquitetura de agentes mediada por MCP para conteúdo tridimensional, com o registro do que ela agrega e do que não agrega; e (iv) um conjunto de evidência visual comparativa organizado, com resultado registrado por critério, incluindo os desvios.

8 CONCLUSÃO

8.1 Retomada do problema

O trabalho partiu da seguinte pergunta: é possível produzir, de forma integralmente reproduzível e automatizada por um agente conectado via Model Context Protocol, um modelo tridimensional de um automóvel esportivo cuja geometria e materiais sejam fiéis a referências fotográficas, atendendo simultaneamente a restrições de topologia de malha e a um envelope dimensional verificado?

A resposta que os resultados sustentam é *sim, com ressalvas precisamente localizadas*. O modelo existe, é reconstruível por um único comando, tem topologia integralmente quadrangular nos corpos principais, envelope dentro da tolerância declarada e os elementos de identidade da variante presentes e verificáveis. A fidelidade de silhueta e de tom de cor, porém, é parcial, e as causas de cada desvio foram identificadas.

8.2 Respostas aos objetivos específicos

O Quadro 8.1 retoma os sete objetivos específicos e o resultado de cada um, com a evidência correspondente.

#	Objetivo	Status	Evidência
1	Extrair dimensões e proporções das referências	Atendido	Tabela 4.1 com origem declarada
2	Gerar casco por <i>loft</i> sem booleanas, sem n-gons	Atendido	750 quads; 0 n-gons
3	Modelar os elementos de identidade do 111S	Atendido	Figuras 6.6 e 6.3
4	Construir biblioteca de materiais fisicamente plausível	Atendido	13 materiais; Figura 6.5
5	Estabelecer a cadeia de automação via MCP	Atendido	build e renders executados por script
6	Emitir relatório de malha a cada build	Atendido	<code>mesh_report.txt</code>
7	Comparar renders com as referências e registrar aceite	Atendido	Quadro 6.3

Fonte: elaborado pelo autor (2026).

Quadro 8.1: Objetivos específicos, resultado e evidência

Os sete objetivos foram atendidos quanto ao método e à evidência. Os critérios parciais (CA-004 e CA-007) não decorrem de objetivos não cumpridos, e sim de que a *qualidade* exigida em dois critérios perceptuais não foi integralmente alcançada.

8.3 Conclusão geral

Três conclusões podem ser afirmadas com o suporte das evidências apresentadas.

Primeira: as restrições topológicas devem ser impostas na construção.

A ausência total de n-gons e de arestas não variedade em uma cena de 32.632 faces foi obtida sem qualquer etapa de reparo. O método de tampa por leque dobrado e a estrutura de encadeamento por anéis são a razão disso. A implicação é direta: a qualidade estrutural de um modelo procedimental pode ser uma *propriedade do gerador*, com garantia mais forte que a de qualquer inspeção posterior.

Segunda: a verificação por medição e a verificação por inspeção são complementares. O relatório numérico confirmou o envelope, a topologia e a ausência de artefatos estruturais, mas foi incapaz de detectar a ausência da linha de caráter. A inspeção visual comparativa identificou esse desvio, mas não teria detectado um n-gon oculto ou um desvio de milímetros no envelope. Concluir que qualquer uma das duas bastaria seria um erro metodológico.

Terceira: a mediação por MCP agrega auditabilidade, com um limite explícito. Cada operação fica serializada como mensagem do protocolo, o que transforma a sessão de modelagem em um registro inspecionável. O limite é que a inspeção visual mediada por captura bidimensional é inferior à inspeção interativa, e que o agente não substitui o julgamento de engenharia sobre o que medir, o que aceitar e o que perseguir.

Do ponto de vista do critério final de um trabalho de engenharia — a existência de um artefato defensável, verificável e rastreável — o resultado é satisfatório dentro dos limites declarados. O modelo não é indistinguível de uma fotografia em *close* de superfície; é, porém, integralmente reconstruível, medido, e cada uma de suas feições é rastreável até uma decisão documentada.

8.4 Trabalhos futuros

As pendências identificadas durante o trabalho definem uma agenda de seis itens, em ordem de relação custo-benefício.

- 1) **Anéis de proximidade na linha de caráter e nos *lips* de para-lama.** É a correção de maior impacto, porque ataca diretamente o critério CA-004 e utiliza uma técnica já validada no próprio projeto (Seção 5.4.4). Alternativamente, avaliar peso de dobra por aresta.
- 2) **Cadeia colorimétrica controlada.** Capturar uma referência com alvo de cor conhecido e aplicar o mesmo mapeamento de tons ao render e à referência, permitindo que CA-007 deixe de ser um julgamento e passe a ser uma medição de diferença de cor.

- 3) **Glifos de contorno fechado nos emblemas.** Substituir o texto convertido por geometria extrusada de contorno fechado, o que elimina as 1.776 arestas de borda e os triângulos de chanfro nos glifos.
- 4) **Validação de continuidade por mapas de zebra e isofotas.** Produzir renders específicos com ambiente de faixas e analisar a continuidade das isofotas, convertendo a alegação de continuidade em evidência.
- 5) **Rig de iluminação de estúdio completo.** Implementar os *softboxes*, os *light cards* longos e o ambiente de alta faixa dinâmica previstos na especificação original, para renders de apresentação de nível comercial.
- 6) **Validação cruzada por reconstrução densa.** Empregar *Structure-from-Motion* ou fotogrametria sobre as mesmas fotografias para obter um envelope independente e compará-lo ao modelo procedimental, atacando a principal ameaça de validade interna.

Além desses itens, dois desdobramentos de pesquisa parecem promissores. O primeiro é generalizar o gerador para uma família de veículos, avaliando se a parametrização por estações se sustenta quando a morfologia varia substancialmente. O segundo é estudar sistematicamente o efeito da mediação por agentes sobre a qualidade do artefato, com avaliação por múltiplos avaliadores e comparação controlada contra um fluxo manual — precisamente o controle experimental que este trabalho, por limitação de escopo, não executou.

8.5 Considerações finais

Este trabalho começou com uma cena vazia e seis fotografias. Termina com um modelo de 130 objetos, cuja geometria é descrita por 1.200 linhas de código que qualquer pessoa pode ler, executar e contestar. Esse é, talvez, o resultado mais duradouro: não o modelo em si, que envelhecerá com a versão do *software*, mas a demonstração de que a modelagem tridimensional pode ser conduzida como engenharia — com especificação, medição, evidência e registro do desvio.

REFERÊNCIAS BIBLIOGRÁFICAS

- [1] Mario Botsch, Leif Kobbelt, Mark Pauly, Pierre Alliez, and Bruno Lévy. *Polygon Mesh Processing*. A K Peters/CRC Press, Boca Raton, 2010.
- [2] Anthropic. Model Context Protocol Specification. Disponível em: <https://modelcontextprotocol.io/specification>, 2024. Acesso em: 13 set. 2026.
- [3] JSON-RPC Working Group. JSON-RPC 2.0 Specification. Disponível em: <https://www.jsonrpc.org/specification>, 2010. Acesso em: 13 set. 2026.
- [4] Brent Burley. Physically Based Shading at Disney. In *SIGGRAPH 2012 Course: Practical Physically Based Shading in Film and Game Production*, 2012.
- [5] Brian Karis. Real Shading in Unreal Engine 4. In *SIGGRAPH 2013 Course: Physically Based Shading in Theory and Practice*, 2013.
- [6] Jami J. Shah and Martti Mäntylä. *Parametric and Feature-Based CAD/CAM: Concepts, Techniques, and Applications*. John Wiley & Sons, New York, 1995.
- [7] Les Piegl and Wayne Tiller. *The NURBS Book*. Springer, Berlin, 2 edition, 1997.
- [8] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [9] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large Language Model Connected with Massive APIs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [10] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [11] Gerhard Pahl, Wolfgang Beitz, Jörg Feldhusen, and Karl-Heinrich Grote. *Engineering Design: A Systematic Approach*. Springer, London, 3 edition, 2007.
- [12] Kevin N. Otto and Kristin L. Wood. *Product Design: Techniques in Reverse Engineering and New Product Development*. Prentice Hall, Upper Saddle River, 2001.

- [13] Mark Halstead, Michael Kass, and Tony DeRose. Efficient, fair interpolation using Catmull-Clark surfaces. In *Proceedings of SIGGRAPH '93*, pages 35–44. ACM, 1993.
- [14] Gerald Farin. *Curves and Surfaces for CAGD: A Practical Guide*. Morgan Kaufmann, San Francisco, 5 edition, 2002.
- [15] Michael E. Mortenson. *Geometric Modeling*. John Wiley & Sons, New York, 2 edition, 1997.
- [16] Jos Stam. Exact evaluation of Catmull-Clark subdivision surfaces at arbitrary parameter values. In *Proceedings of SIGGRAPH '98*, pages 395–404. ACM, 1998.
- [17] Edwin Catmull and James Clark. Recursively generated B-spline surfaces on arbitrary topological meshes. *Computer-Aided Design*, 10(6):350–355, 1978.
- [18] Charles Loop. Smooth Subdivision Surfaces Based on Triangles. Master’s thesis, University of Utah, Salt Lake City, 1987.
- [19] Blender Foundation. Blender Manual. Disponível em: <https://docs.blender.org/manual/en/latest/>, 2026. Acesso em: 13 set. 2026.
- [20] James T. Kajiya. The rendering equation. In *Proceedings of SIGGRAPH '86*, pages 143–150. ACM, 1986.
- [21] Matt Pharr, Wenzel Jakob, and Greg Humphreys. *Physically Based Rendering: From Theory to Implementation*. Morgan Kaufmann, Cambridge, MA, 3 edition, 2016.
- [22] Bui Tuong Phong. Illumination for computer generated pictures. volume 18, pages 311–317, 1975.
- [23] James F. Blinn. Models of light reflection for computer synthesized pictures. In *Proceedings of SIGGRAPH '77*, pages 192–198. ACM, 1977.
- [24] Robert L. Cook and Kenneth E. Torrance. A reflectance model for computer graphics. *ACM Transactions on Graphics*, 1(1):7–24, 1982.
- [25] Bruce Walter, Stephen R. Marschner, Hongsong Li, and Kenneth E. Torrance. Microfacet Models for Refraction through Rough Surfaces. In *Proceedings of the Eurographics Symposium on Rendering (EGSR)*, pages 195–206, 2007.
- [26] Christophe Schlick. An Inexpensive BRDF Model for Physically-based Rendering. *Computer Graphics Forum*, 13(3):233–246, 1994.
- [27] Michael Ashikhmin and Peter Shirley. An anisotropic Phong BRDF model. *Journal of Graphics Tools*, 5(2):25–32, 2000.

- [28] Gregory J. Ward. Measuring and modeling anisotropic reflection. In *Proceedings of SIGGRAPH '92*, pages 265–272. ACM, 1992.
- [29] Erik Reinhard, Michael Stark, Peter Shirley, and James Ferwerda. Photographic tone reproduction for digital images. *ACM Transactions on Graphics*, 21(3):267–276, 2002.
- [30] Mark D. Fairchild. *Color Appearance Models*. John Wiley & Sons, Chichester, 3 edition, 2013.
- [31] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [32] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, et al. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [33] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [34] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [35] Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, Cambridge, 2 edition, 2004.
- [36] George Stiny and James Gips. Shape grammars and the generative specification of painting and sculpture. In *Information Processing 71*, pages 1460–1465. North-Holland, Amsterdam, 1972.
- [37] Przemyslaw Prusinkiewicz and Aristid Lindenmayer. *The Algorithmic Beauty of Plants*. Springer, New York, 1990.
- [38] Yoav I. H. Parish and Pascal Müller. Procedural modeling of cities. In *Proceedings of SIGGRAPH '01*, pages 301–308. ACM, 2001.
- [39] Pascal Müller, Peter Wonka, Simon Haegler, Andreas Ulmer, and Luc Van Gool. Procedural modeling of buildings. *ACM Transactions on Graphics*, 25(3):614–623, 2006.
- [40] Jon McCormack, Alan Dorin, and Troy Innocent. Generative Design: a paradigm for design research. In *Proceedings of FutureGround, Design Research Society*, 2004.

- [41] Takeo Igarashi, Satoshi Matsuoka, and Hidehiko Tanaka. Teddy: a sketching interface for 3D freeform design. In *Proceedings of SIGGRAPH '99*, pages 409–416. ACM, 1999.
- [42] Tamás Várady, Ralph R. Martin, and Jordan Cox. Reverse engineering of geometric models—an introduction. *Computer-Aided Design*, 29(4):255–268, 1997.
- [43] Noah Snavely, Steven M. Seitz, and Richard Szeliski. Photo Tourism: Exploring Photo Collections in 3D. In *ACM Transactions on Graphics*, volume 25, pages 835–846, 2006.
- [44] Johannes L. Schönberger and Jan-Michael Frahm. Structure-from-Motion Revisited. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4104–4113, 2016.
- [45] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2020.
- [46] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM Transactions on Graphics*, 42(4), 2023.
- [47] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *International Conference on Learning Representations (ICLR)*, 2024.
- [48] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155, 2023.
- [49] blender-mcp contributors. blender-mcp: Blender integration server for the Model Context Protocol. Software. Disponível em: <https://github.com/ahujasid/blender-mcp>, 2025. Acesso em: 13 set. 2026.

A TABELA COMPLETA DAS ESTAÇÕES DO CASCO

A Tabela A.1 reproduz as 47 estações transversais que definem o casco. As colunas w , w_t , z_s , z_{te} e z_t são parâmetros declarados no código; a coluna z_b é *calculada* pela função de arco de roda, e por isso aparece aqui como valor derivado. As estações 1–2, 31–32 e 46–47 são pares de retenção inseridos para estabilizar as extremidades sob subdivisão. As estações 11–19 e 33–39 delimitam os vãos de roda dianteiro e traseiro, respectivamente.

Tabela A.1: Tabela completa das estações do casco (metros)

#	x	w	w_t	z_b	z_s	z_{te}	z_t
1	1.863	0.625	0.500	0.190	0.470	0.560	0.592
2	1.856	0.632	0.506	0.189	0.472	0.562	0.594
3	1.848	0.712	0.580	0.187	0.490	0.578	0.610
4	1.806	0.775	0.630	0.180	0.512	0.600	0.632
5	1.740	0.792	0.646	0.168	0.530	0.618	0.650
6	1.680	0.802	0.656	0.158	0.545	0.632	0.664
7	1.610	0.806	0.660	0.145	0.558	0.646	0.678
8	1.545	0.808	0.660	0.134	0.570	0.658	0.690
9	1.530	0.808	0.660	0.131	0.572	0.660	0.692
10	1.520	0.808	0.660	0.128	0.574	0.662	0.694
11	1.470	0.810	0.660	0.391	0.590	0.676	0.708
12	1.380	0.814	0.660	0.536	0.618	0.700	0.730
13	1.280	0.818	0.660	0.601	0.648	0.720	0.748
14	1.200	0.820	0.660	0.623	0.668	0.735	0.760
15	1.150	0.820	0.660	0.627	0.680	0.742	0.766
16	1.100	0.818	0.660	0.623	0.676	0.738	0.762
17	1.020	0.812	0.655	0.601	0.656	0.726	0.754
18	0.930	0.806	0.650	0.545	0.634	0.716	0.750
19	0.860	0.800	0.642	0.460	0.616	0.712	0.752
20	0.790	0.795	0.636	0.128	0.600	0.706	0.748
21	0.775	0.794	0.634	0.128	0.596	0.704	0.746
22	0.700	0.792	0.630	0.128	0.592	0.700	0.742
23	0.550	0.790	0.624	0.128	0.598	0.700	0.740
24	0.380	0.790	0.622	0.128	0.610	0.702	0.740
25	0.200	0.793	0.624	0.128	0.626	0.706	0.744

#	x	w	w_t	z_b	z_s	z_{te}	z_t
26	0.020	0.797	0.626	0.128	0.640	0.712	0.748
27	-0.160	0.803	0.630	0.128	0.652	0.720	0.756
28	-0.340	0.815	0.638	0.128	0.670	0.762	0.816
29	-0.520	0.832	0.648	0.128	0.692	0.812	0.872
30	-0.680	0.848	0.654	0.128	0.710	0.856	0.900
31	-0.755	0.852	0.656	0.128	0.716	0.866	0.906
32	-0.765	0.852	0.656	0.128	0.716	0.866	0.906
33	-0.850	0.855	0.656	0.486	0.720	0.870	0.908
34	-0.950	0.859	0.657	0.593	0.726	0.872	0.910
35	-1.050	0.859	0.657	0.641	0.734	0.874	0.911
36	-1.150	0.859	0.657	0.656	0.740	0.874	0.911
37	-1.250	0.858	0.656	0.641	0.740	0.872	0.909
38	-1.350	0.855	0.652	0.593	0.736	0.868	0.905
39	-1.450	0.850	0.646	0.486	0.726	0.860	0.897
40	-1.545	0.843	0.638	0.205	0.714	0.850	0.886
41	-1.558	0.841	0.636	0.210	0.710	0.846	0.882
42	-1.640	0.836	0.632	0.239	0.700	0.836	0.870
43	-1.720	0.822	0.618	0.268	0.686	0.818	0.848
44	-1.780	0.802	0.600	0.290	0.660	0.790	0.814
45	-1.820	0.775	0.570	0.304	0.634	0.752	0.778
46	-1.852	0.735	0.520	0.316	0.606	0.706	0.732
47	-1.858	0.735	0.520	0.318	0.606	0.706	0.732

Fonte: extraído de *BODY_STATIONS*; valores de z_b calculados por *arch_bottom* (2026).

B INVENTÁRIO DE MATERIAIS

A Tabela B.1 consolida os treze materiais da cena e os parâmetros declarados no código. Cores são apresentadas no valor sRGB de origem, antes da conversão para linear.

Tabela B.1: Materiais da cena e parâmetros principais

Material	Cor sRGB	Rugosidade	Particularidades
MAT_Blue_Metallic	#0C1A96	base baixa	verniz: peso 0,6, rugosidade 0,02, IOR 1,55; flocos por ruído; profundidade por peso de camada
MAT_Glass_Polycarbonate	claro	0,02	transmissão 1,0; IOR 1,58; refração por raio
MAT_Headlamp_Lens	claro	0,03	transmissão 1,0; IOR 1,52
MAT_Lens_Red	vermelho	0,05	transmissão 1,0
MAT_Lens_Amber	âmbar	0,05	transmissão 1,0
MAT_Rubber_Tire	preto	0,85	ruído em relevo
MAT_Aluminum_Brushed	alumínio	0,32	metálico 1,0; anisotropia 0,7; ruído
MAT_Chrome_Trim	cromado	0,08	metálico 1,0
MAT_Badge_Chrome	cromado	0,06	metálico 1,0
MAT_Reflector_Chrome	cromado	0,10	metálico 1,0
MAT_Black_Trim	preto	0,55	ruído leve
MAT_Grille_Mesh	#17191B	0,65	preto escuro
MAT_Interior_Fabric	preto	0,90	ruído

Fonte: elaborado pelo autor a partir de `build_materials` (2026). Valores de rugosidade arredondados conforme declarados no código.

Observação de integridade: o material de material neutro usado nas vistas de verificação (MAT_Clay_Check) não pertence ao modelo. Ele é criado temporariamente pelo script de verificação como sobreposição de material da camada de visualização, e é removido ao final, de modo que não aparece no inventário do modelo entregue.

C INVENTÁRIO DE OBJETOS E RELATÓRIO DE MALHA

Esta seção reproduz os totais e o envelope do relatório de malha emitido pelo build. A tabela detalhada, com uma linha por objeto e as contagens de vértices, faces, quads, triângulos, n-gons, arestas não variedade e arestas de borda, está no arquivo `docs/renders/mesh_report.txt`, que é gerado automaticamente e versionado.

C.1 Totais consolidados

Tabela C.1: Totais de malha da cena (malha base, sem subdivisão)

Grandeza	Valor
Objetos de malha	130
Faces	32.632
quads	29.272
triângulos	3.360
n-gons	0
Arestas não variedade	0
Arestas de borda	1.776

Fonte: docs/renders/mesh_report.txt (2026).

C.2 Envelope dimensional

Tabela C.2: Envelope medido e alvo

Grandeza	Medido (m)	Alvo (m)	Observação
Comprimento (X)	3,736	3,726	+1,0 cm
Largura (Y)	1,718	1,718	exato
Altura (Z)	1,117	1,117	exato
Solo (Z mínimo)	0,0000	0,000	exato
Centro em Y	0,0000	0,000	exato

Fonte: docs/renders/mesh_report.txt (2026).

Tabela C.3: Objetos de malha por grupo funcional

Grupo	Objetos
Rodas (4 conjuntos de 13 peças)	52
Ventilações (<i>louvers</i> , saídas, duto)	23
Faróis e indicadores	12
Lanternas traseiras	8
Interior	7
Grelha frontal	6
Aerodinâmica	6
Faróis auxiliares	4
Espelhos	4
Casco e cabine	3
Emblemas	3
Refletores do difusor	2
Total	130

Fonte: docs/renderers/mesh_report.txt (2026).

C.3 Inventário por grupo funcional

C.4 Objetos com arestas de borda

Apenas três objetos apresentam arestas de borda, todos eles emblemas de texto: GEO_Badge_LOTUS (1.240), GEO_Badge_111S (536) e, adicionalmente, as bordas de contorno dos demais glifos. Nenhum objeto de superfície curva apresenta aresta de borda, o que confirma que os corpos são fechados e as tampas operam conforme projetado.

C.5 Objetos com triângulos

Os triângulos concentram-se em três origens: os glifos dos emblemas (GEO_Badge_LOTUS com 604 e GEO_Badge_111S com 252), os cantos de chanfro de primitivas de caixa e os elementos ópticos de revolução. Nenhum triângulo ocorre em superfície curva principal do casco ou da cabine.

D MÓDULOS, EXECUÇÃO E REPRODUTIBILIDADE

D.1 Estrutura de *software*

Tabela D.1: Módulos do projeto

Arquivo	Linhas	Responsabilidade
scripts/lotus_common.py	≈ 570	helpers de geometria, materiais, cores e medição; sem efeitos colaterais
scripts/build_lotus_elise_111s.py	≈ 1.230	constrói a cena, salva o <code>.blend</code> e emite o relatório
scripts/render_views.py	≈ 130	nove renders de verificação, incluindo duas vistas em material neutro
scripts/render_gallery.py	≈ 250	cinco imagens de apresentação, com rig dramático temporário
scripts/mesh_report.py	≈ 90	relatório de malha avulso

Fonte: elaborado pelo autor (2026). As contagens de linhas são aproximadas.

D.2 Comandos de reconstrução

A reconstrução integral do artefato requer apenas o Blender em versão compatível e o repositório. Os comandos, na ordem:

```
1 # 1. construir o modelo (limpa a cena e reconstrói)
2 blender --background car-lotus-elise-111s.blend \
3     --python scripts/build_lotus_elise_111s.py
4 # 2. renders de verificacao
5 blender --background car-lotus-elise-111s.blend \
6     --python scripts/render_views.py
7 # 3. galeria de apresentacao
8 blender --background car-lotus-elise-111s.blend \
9     --python scripts/render_gallery.py
10 # 4. relatorio de malha avulso
11 blender --background car-lotus-elise-111s.blend \
12     --python scripts/mesh_report.py
```

Alternativamente, na sessão viva do Blender mediada por MCP:

```
1 exec(compile(open("scripts/build_lotus_elise_111s.py").read(),
2     "build_lotus.py", "exec"))
```

D.3 Condições de reprodutibilidade

Para que os resultados do Capítulo 6 sejam reproduzidos, as seguintes condições devem valer:

- 1) Blender em versão compatível com 5.2.1 LTS, com o motor EEVEE disponível.
- 2) Presença das pastas `scripts/` e `images/` na raiz do projeto, exigidas pela função de detecção de raiz.
- 3) Execução dos scripts na ordem apresentada, partindo de um arquivo de cena existente (o build limpa a cena de qualquer forma).
- 4) Ausência de edições manuais não registradas: o script é a fonte da verdade.

D.4 Artefatos de especificação

Os documentos que acompanham o código estão em `.specs/` e compreendem a constituição do projeto, a especificação com requisitos e critérios de aceitação, o registro de decisões de arquitetura e a lista de tarefas. Além deles, o repositório mantém `README.md`, `STATUS.md` e `HANDOFF.md`, que registram, respectivamente, a apresentação do projeto, o estado de desenvolvimento e a continuidade entre agentes.

D.5 Divergência identificada durante a redação

Durante a redação deste documento, a contagem de estações do casco foi verificada diretamente no código e confrontada com o relatório de malha. A verificação revelou que o casco possui **47** estações, e não 45 como registrado na documentação de acompanhamento do projeto. A evidência decisiva é aritmética: 46 faixas de 16 faces mais duas tampas de 7 quads resultam em 750 faces, exatamente o valor medido no relatório, o que só é consistente com 47 estações. A divergência foi registrada como pendência de documentação e não afeta o modelo, que é determinado pelo código.

Do mesmo modo, a descrição do perfil transversal na documentação de projeto menciona dez pontos e dezoito segmentos, enquanto o código define nove pontos de meio-lado, o que resulta em dezesseis pontos por anel fechado — valor confirmado pela aritmética das faces. Essas divergências não são defeitos do modelo; são desatualizações entre a documentação e o código, e ilustram por que o relatório de malha, e não a documentação textual, é tratado como evidência primária neste trabalho.

A divergência foi comunicada e corrigida nos arquivos de acompanhamento do projeto (`STATUS.md`, `HANDOFF.md`) durante a redação desta monografia; a descrição do perfil permanece a atualizar no registro de decisões de projeto. O episódio é, ele próprio,

um resultado do método: exigir que todo número afirmado tenha uma evidência rastreável expõe inconsistências de documentação que, de outro modo, passariam despercebidas.